Assignment Report

CS6903: Network Security **2025-26 - Semester II**

Name of members- AKARSH DUBEY - CS25MTECH14001
ATISH KADAM - CS25MTECH14003

Task 1: Basic File Management Service

Summary and overview of TASK1

For Task 1, this is remote file management system using a Client-Server model. The aim here is to get two different processes talking to each other over a network. I used standard Python sockets to let a client send commands like LIST or INFO to a server, which then processes them one by one. Here the client acts as the interface for the user, and the server acts as the "manager" for the files stored in a specific directory.

Implementation Approach

To get the basic architecture running, I focused on setting up a stable connection before handling the specific commands:

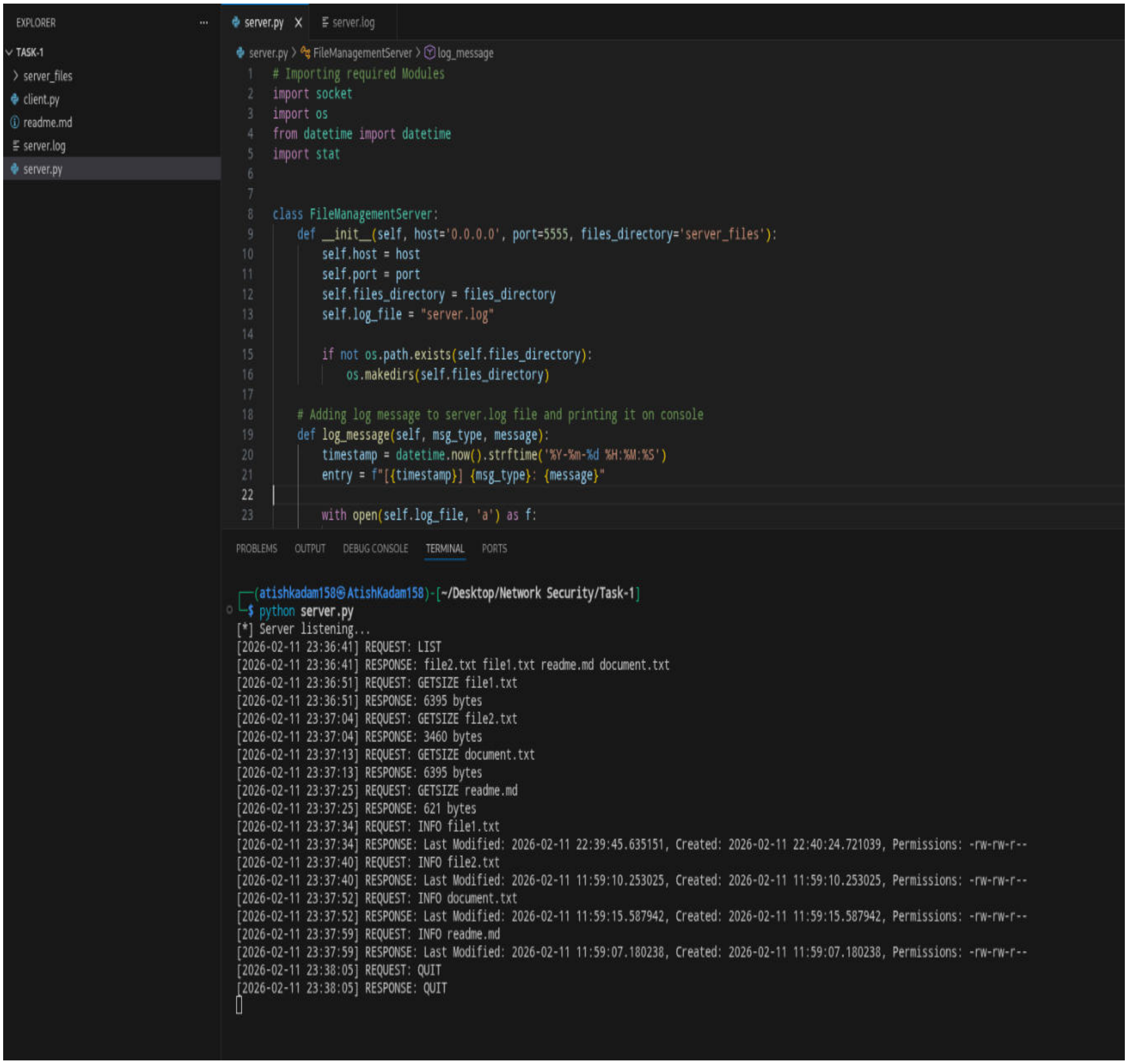
Client Side:
Here I used raw_input.split(maxsplit=1) so the client can handle commands with or without arguments (like just LIST vs INFO file.txt). In this i have used the command part to .upper() to make sure the server recognizes it even if the user types in lowercase.
Server Side:
The server uses an if-elif block inside process_command() to route the request to the right helper function (like handle_list or handle_info).

Socket Backbone: I used the socket library to set up a TCP connection. On the server side, I used setsockopt(SO_REUSEADDR) . This is a life-saver because it lets me restart the server immediately without getting "Address already in use" errors. The server binds to port 5555 and listens for the client's connect() request.
Logging System: I created a log_message function that writes every transaction to server.log. It captures the timestamp, whether it was a "REQUEST" or a "RESPONSE," and the actual data sent. It's set to append mode ('a'), so the history isn't wiped every time the server restarts.

Clean Exit: I implemented a QUIT command that sends a signal to both sides to break out of their loops and close the sockets properly, ensuring no "hanging" connections are left on the OS.

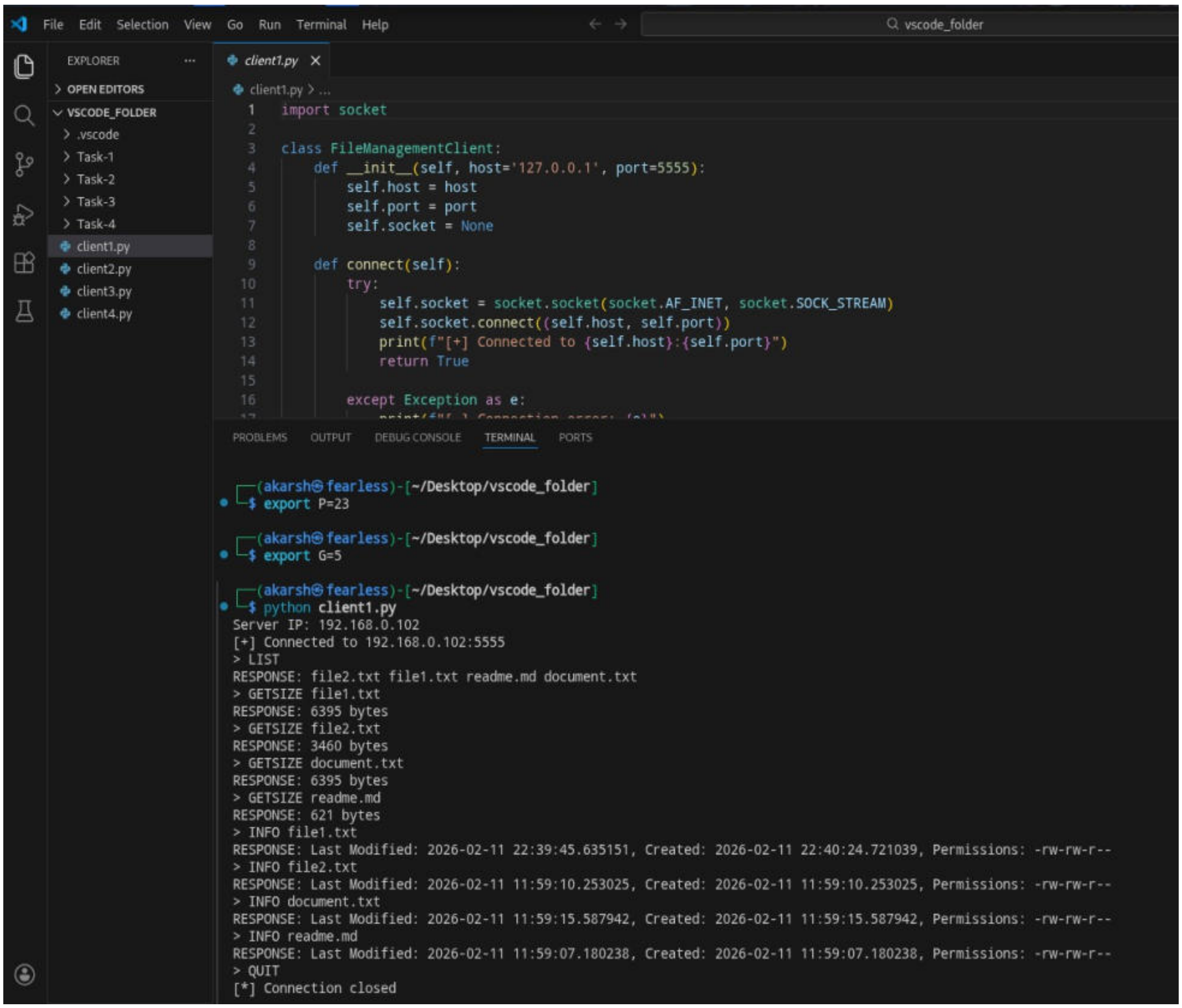
Server Side

This is the Task1 server.py file terminal capture screenshot.
In this screenshot , server Terminal shows REQUEST and RESPONSE of LIST, INFO, GETSIZE, and QUIT commands requested by user alongwith the timestamp.



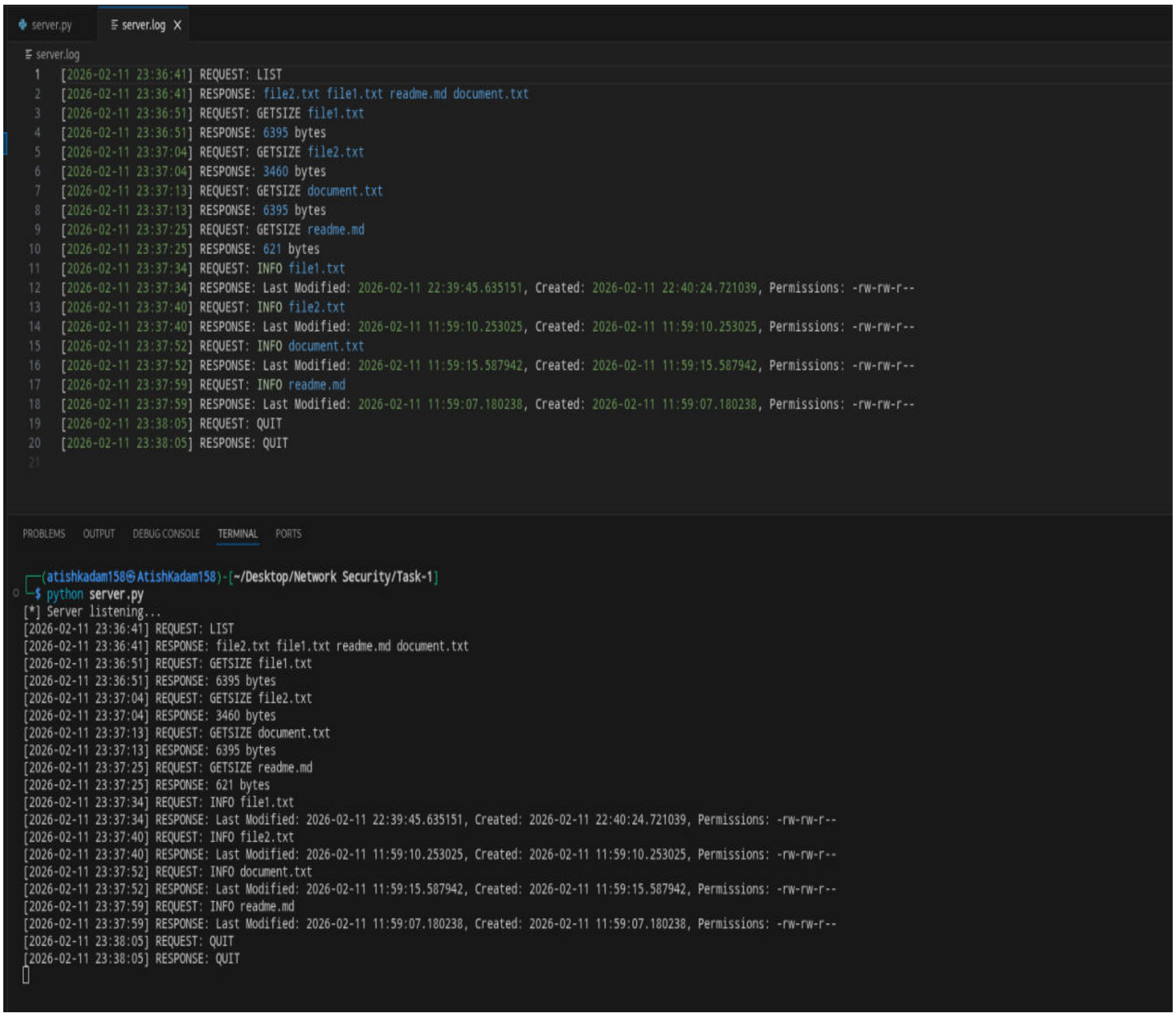
Client side

This is the TASK1 client.py file terminal capture screenshot. In this screenshot , the client got the response of input command given to server in this format as like of LIST command it shows as RESPONSE: file2.txt file1.txt readme.md document.txt .Similarly the client got the response of other commands of GETSIZE,INFO and QUIT .



Server.log

This is the TASK 1 **server.log** file ,which shows about the activity performed at server by client. It contains the timestamp at which the client asked for the command and at what time the response was received at client.



Task 2: Authentication with Multiple Clients and Session Key Establishment

Summary and Design Overview

The objective of this task is to transition from a basic service to a secure, multi-user environment. The design focuses on three core security pillars:
Concurrency: Utilizing threading to handle multiple clients simultaneously.
Authentication: Implementing a Challenge-Response protocol using nonces and SHA-256 to prevent password sniffing.
Key Exchange: Establishing a symmetric session key via the Diffie-Hellman protocol to secure future communications.

Implementation Approach

Based on the submitted code, the implementation follows these steps:

- **Multi-Client Support:** The server uses the threading library. For every successful accept() call, a new thread is spawned to handle that specific client's session, ensuring the server remains responsive to other incoming connections.
- **Authentication Protocol:**
  - The client sends a username.
  - The server checks credentials.txt. If the user exists, it generates a cryptographically secure random **nonce** and sends it to the client.
    The client concatenates the nonce and their shared_secret, then sends the SHA-256 hash back: HASH(nonce \ || \ shared_secret)
  - The server performs the same calculation; if they match, access is granted.

  Diffie-Hellman (DH) Key Exchange:

  - The client pulls P and G from environment variables.
  - Both parties generate private keys (a and b) and exchange public values (A = G^a \mod P and B = G^b \mod P).
  - The final shared secret is derived as K = B^a \mod P (client) and K = A^b \mod P (server).
- **Account Lockout Policy:** If a username fails three consecutive times, the server logs the event, blocks the username for that session, and closes the socket.

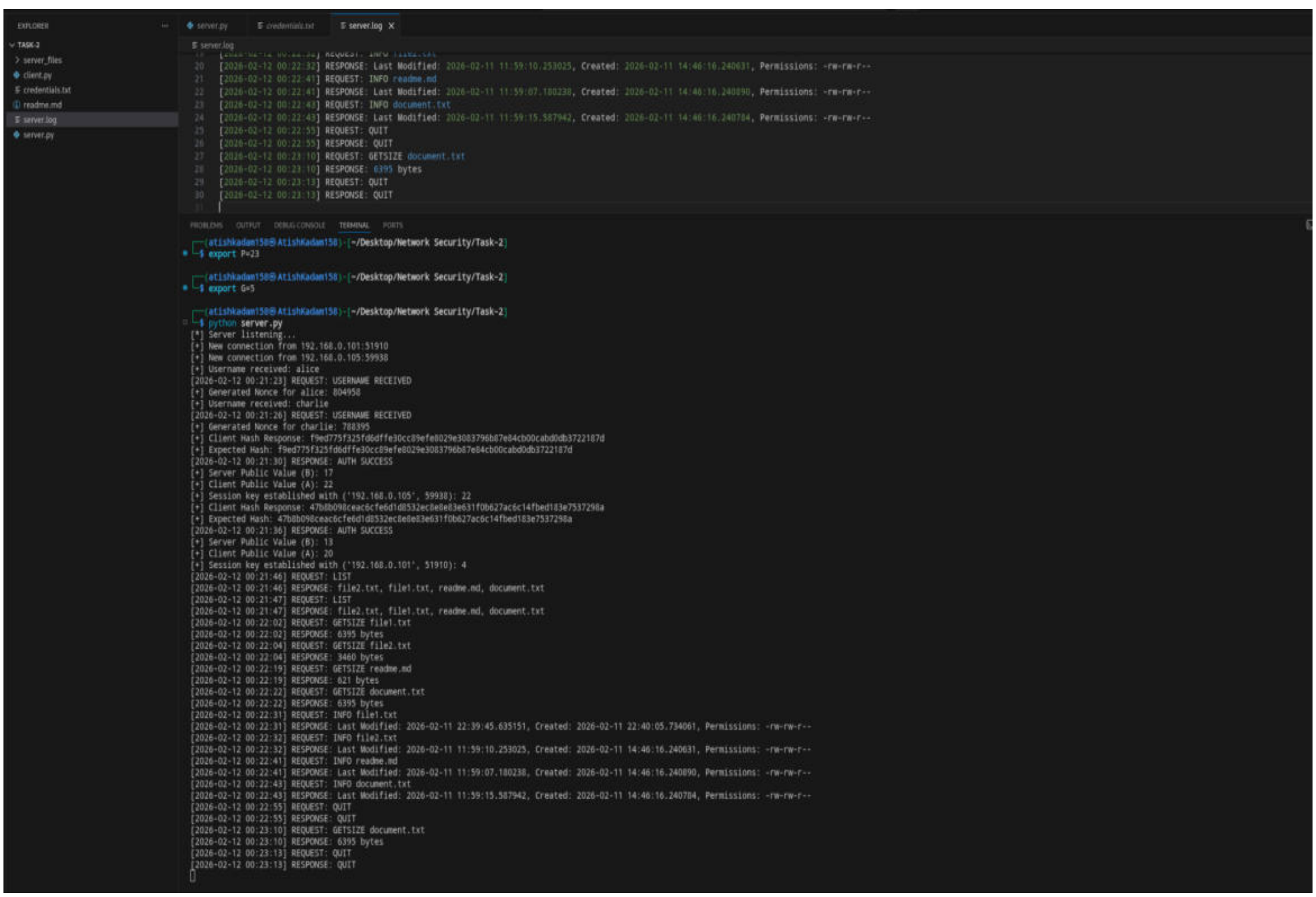
Server.log

This is the **Task2 server.log** file terminal capture screenshot.
In this basically after implementation of TASK1 we have made this task as TASK2 which consist of added authentication ,multi client support as well as secure key establishment.
Here after the matching of username and shared secret ,this shows “**AUTH SUCCESS**” which means that the client1 name “alice” and client2 name “charlie” is now authenticated to server .

Later in order to establish the session ,
the client A public value and server B public value of both the client is computed to make the secret key for the session establishment and
show the output as “**Session key established with (‘192.168.0.105’,59938)** for client2
and “**Session key established with (‘192.168.0.101’, 51910)** for client 1

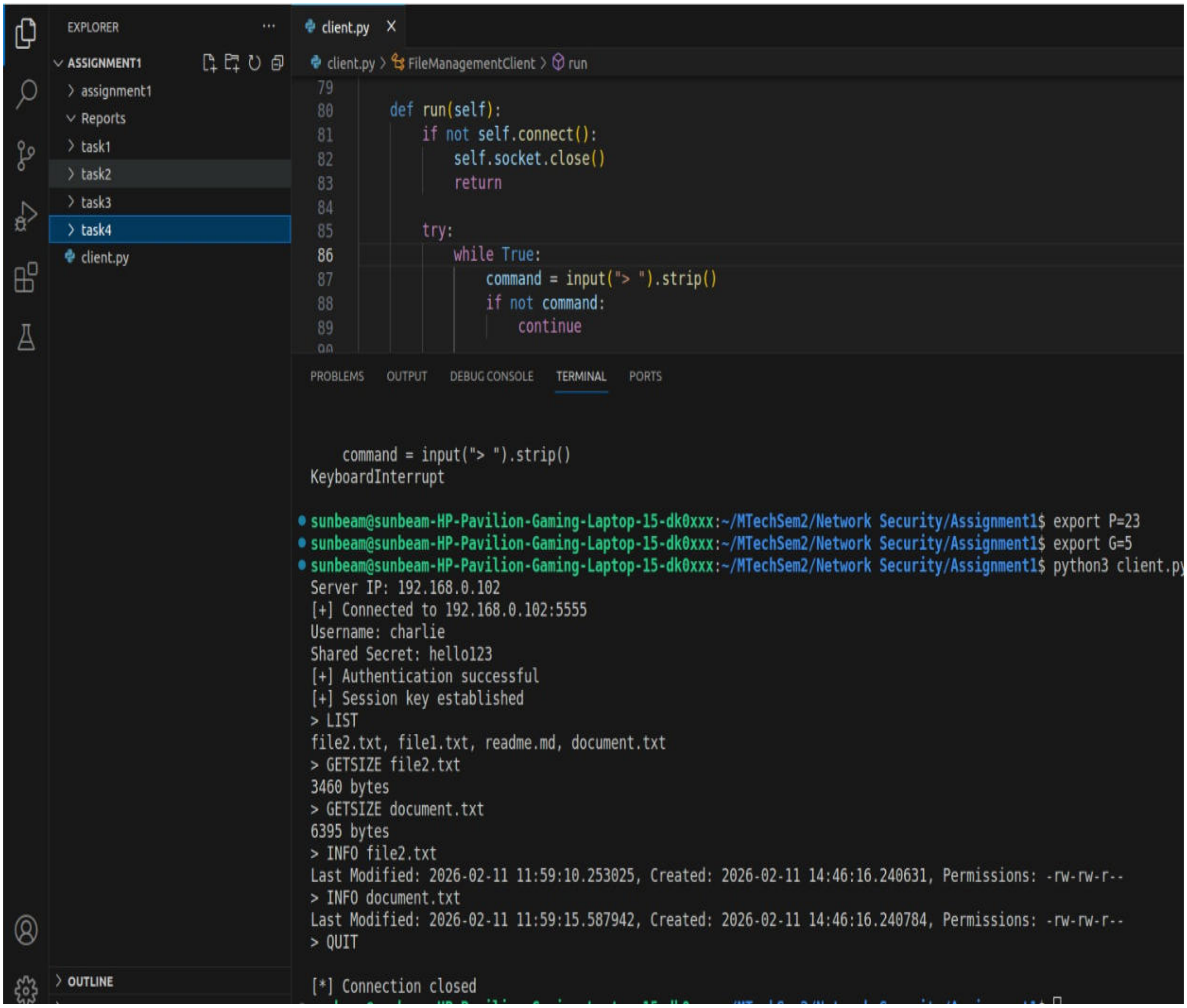
Further this task gives the response asked by client1 and 2 for the commands like LIST, GETSIZE file1.txt, INFO file1.txt, QUIT command same as done in TASK1 but here we have done the same for multiclient (192.168.0.105 , 59938 as client 2)
(192.168.0.101, 51910 as client 1)

Server side



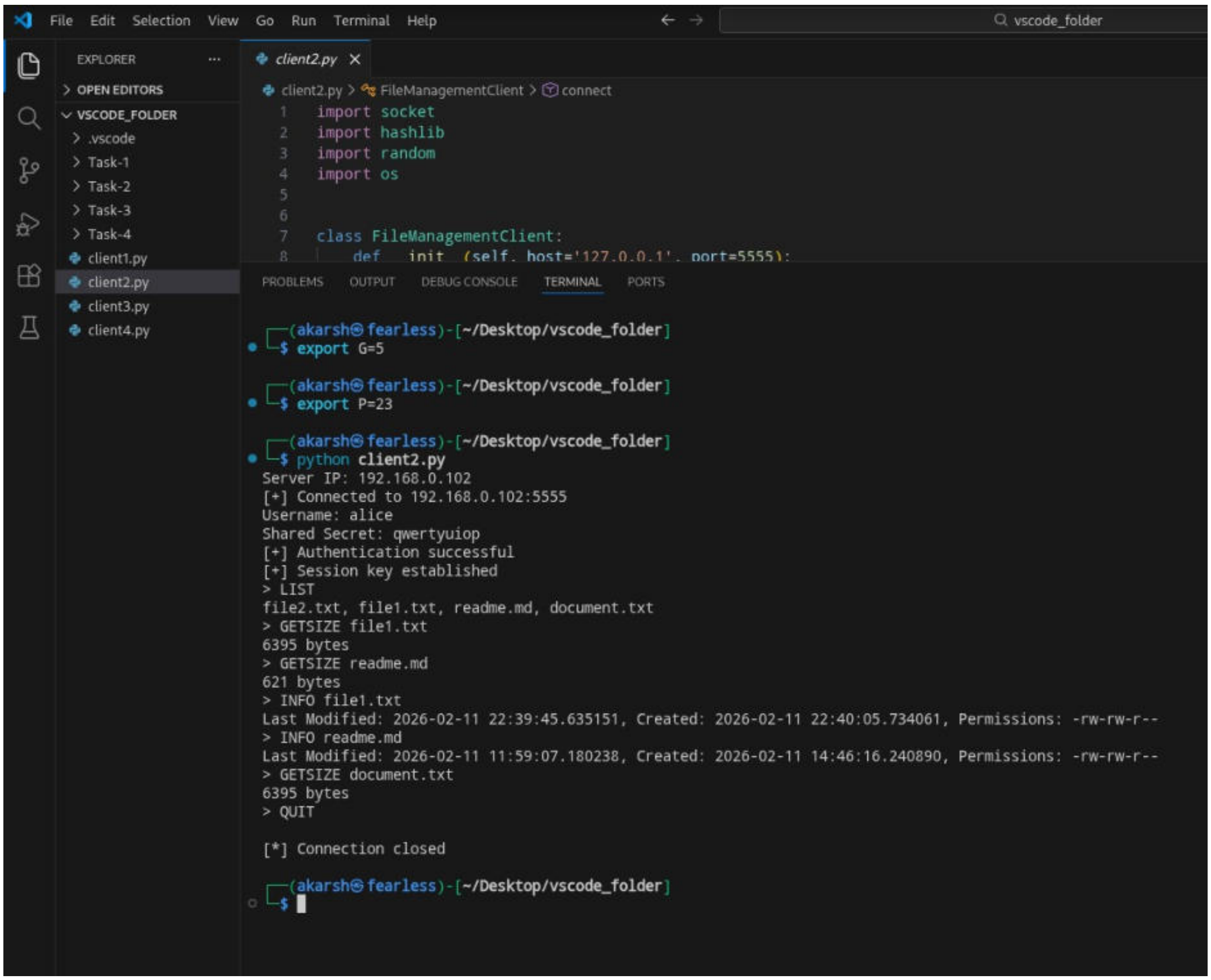
Client 1 side (as multiuser)

This is the capture of Client 1 (as multiclient user) for TASK2 with IP 192.168.0.101 connected with server IP=192.168.0.102'
Here the client 1 (multiuser) is **"charlie"** with it "shared secret key : hello123" has been authenticated by server as we can see in terminal it shows "Authentication Successful" and further the session key established and after which only the client 1 performs the command as LIST,GETSIZE file2.txt etc as it can be seen in the terminal capture below.



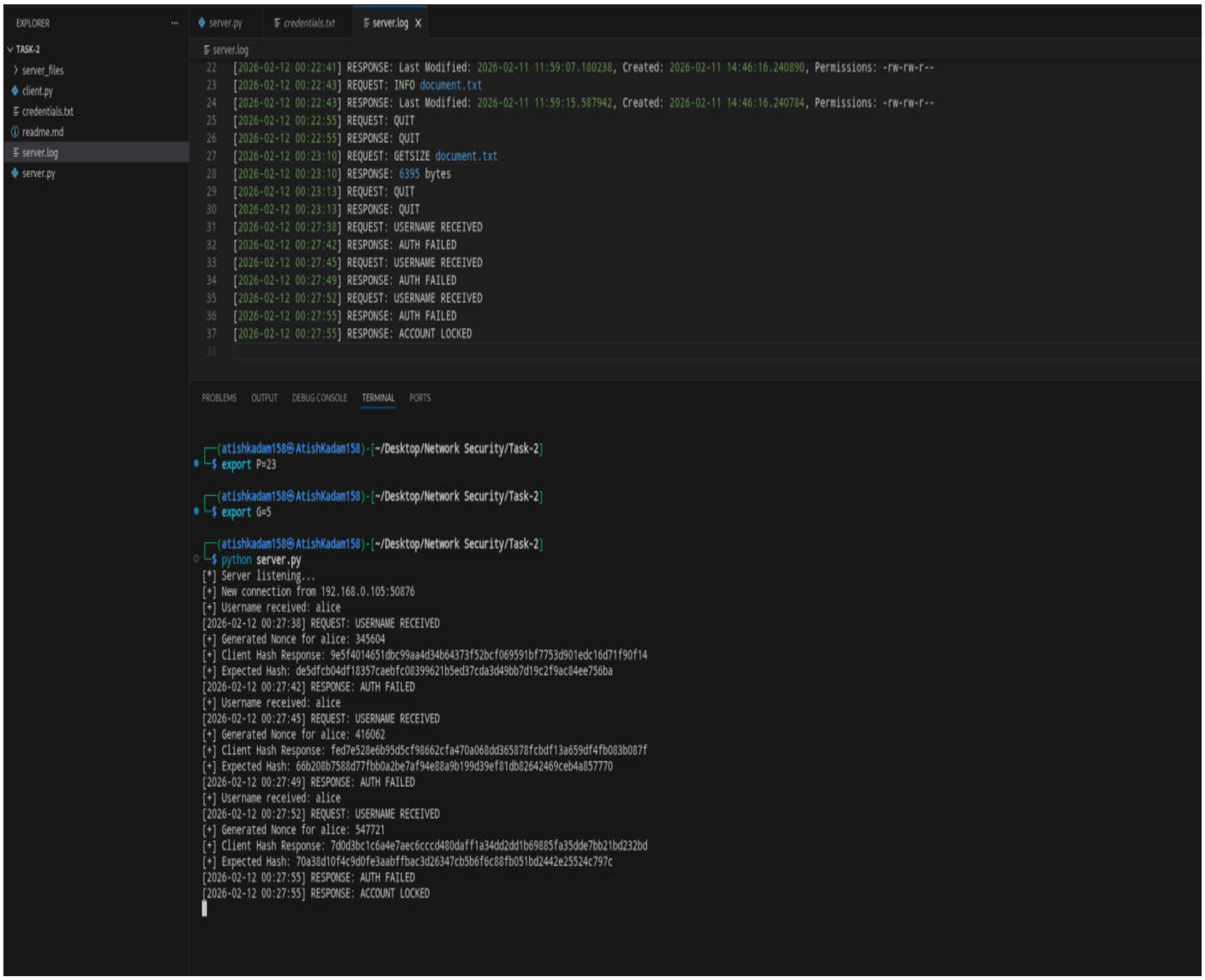
Client 2 side (as multiuser)

This is the capture of Client 2 (as multiclient user) for TASK2 with IP 192.168.0.105 connected with server IP=192.168.0.102
Here the client2 (multiuser) is **“alice”** with it “shared secret key : qwertyuiop” has been authenticated by server as we can see in terminal it shows “Authentication Successful” and further the session key established with server and after which only the client2 performs the command as LIST,GETSIZE file1.txt etc as it can be seen in the terminal capture below.



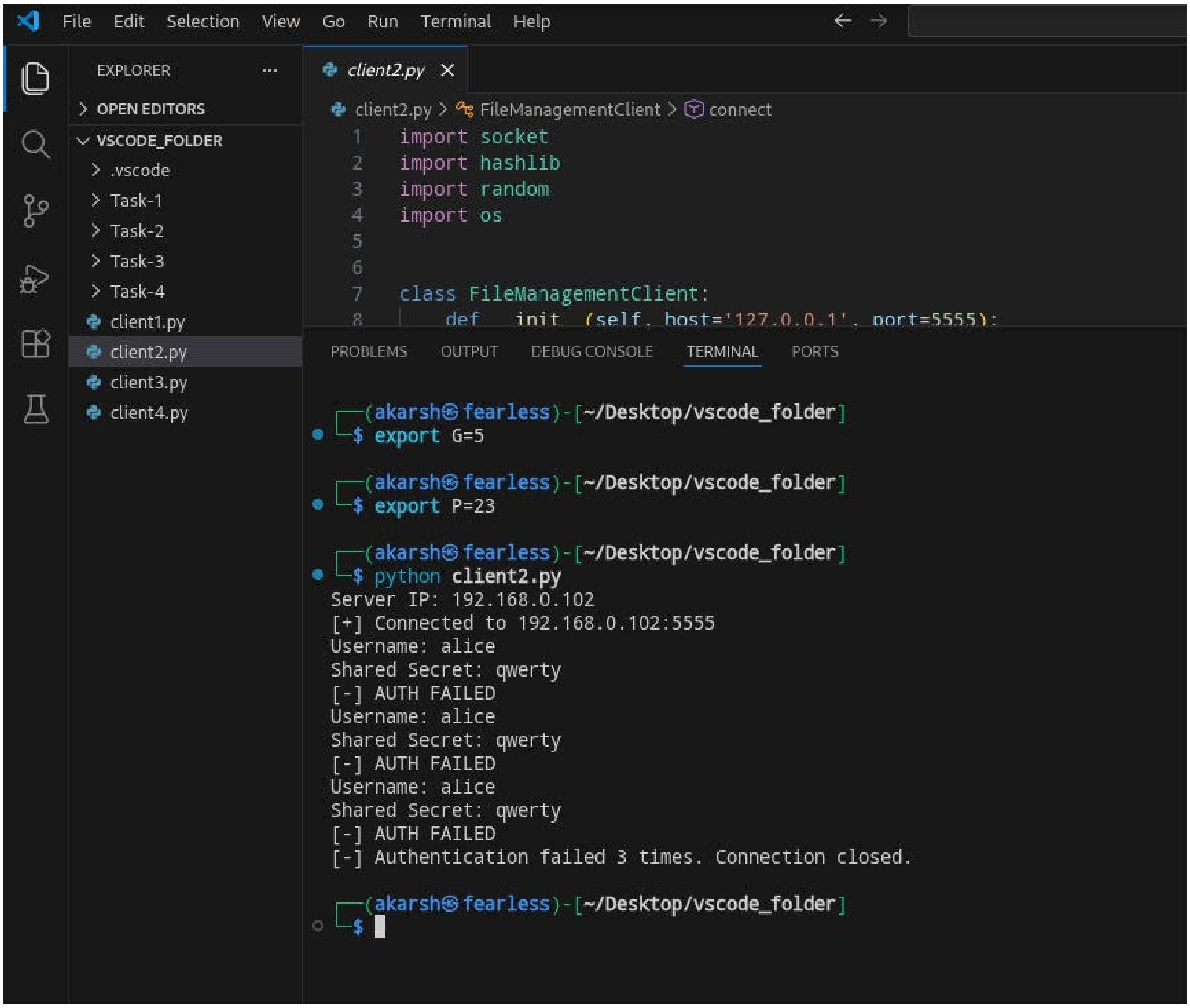
Server side(terminal during lockout)

Now here in **TASK2 of server.log file** terminal capture
As per **Account Lockout Policy** we have demonstrated how the account gets blocked by the client when the credential does not match as of server credential.
Here is the capture of the server terminal when the account gets blocked .
Here the client with IP=192.168.0.105 is blocked by server for entering the wrong shared secret key.



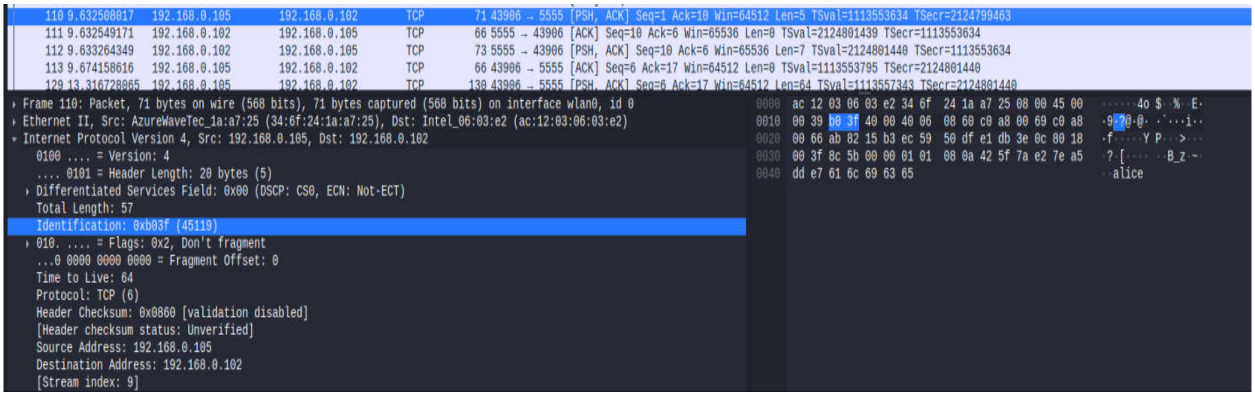
Client side (terminal during lockout)

This is the client2.py file terminal capture
In this it shows how the client with IP =192.168.0.105 name “alice” enters the username on server ip= 192.168.0.102 but enters the wrong shared secret key consecutive 3 times which makes the server to block the account.

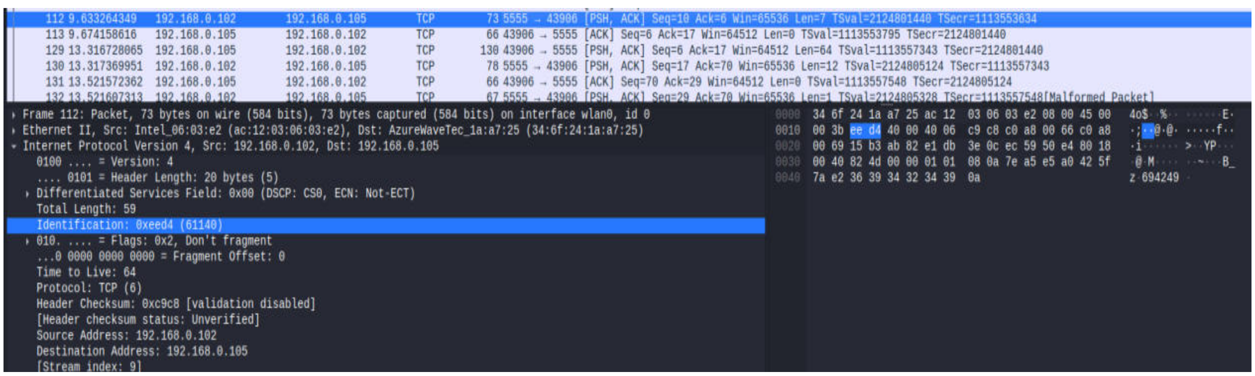


Now here is the Wireshark Captures for TASK 2

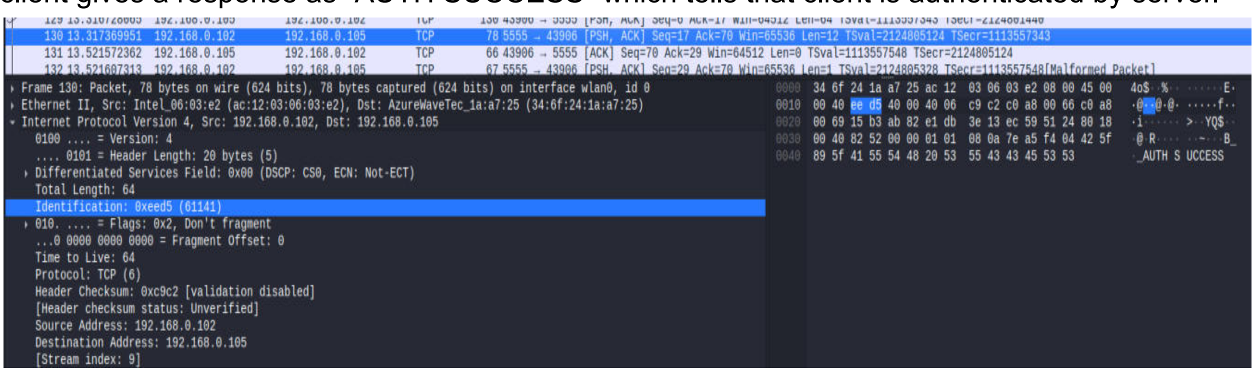
Here the capture shows the client with IP=192.168.0.105 has given its username as “alice” along with a shared secret key to the server IP=192.168.0.102 for authentication.



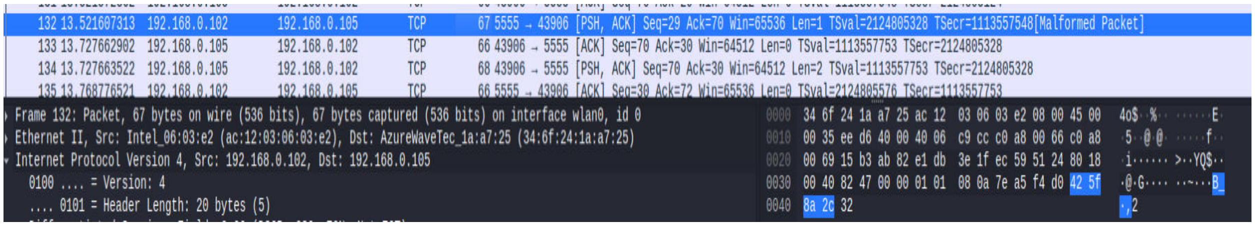
This capture shows the client with IP=192.168.0.105 has generated in the form as HASH (nonce || shared_secret) and give it to server for authentication .

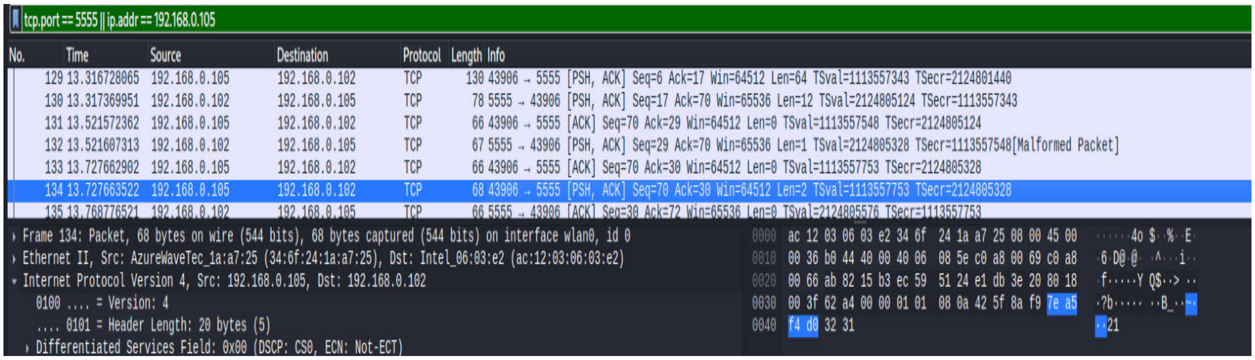


In this capture it shows that   the server (192.168.0.102) after checking the HASH generated by client gives a response as “AUTH SUCCESS” which tells that client is authenticated by server.

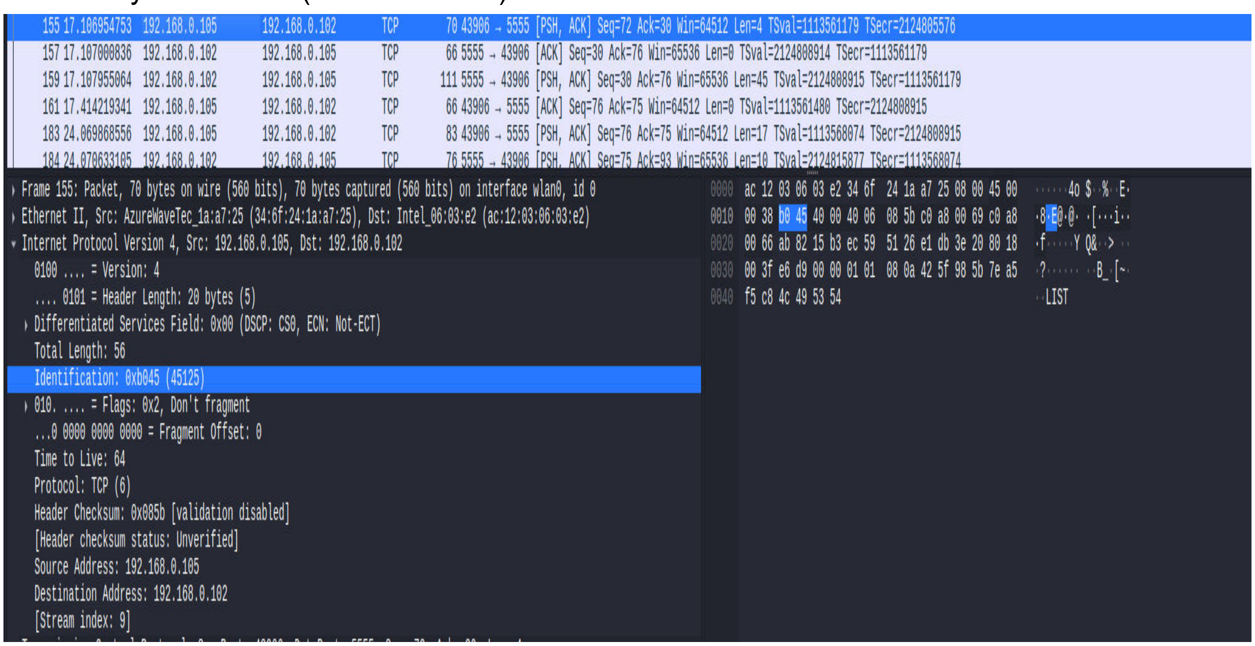


This consecutive capture shows that server(192.168.0.102) is showing the client and server public key value as 2 and 21 respectively is generated for session establishment.

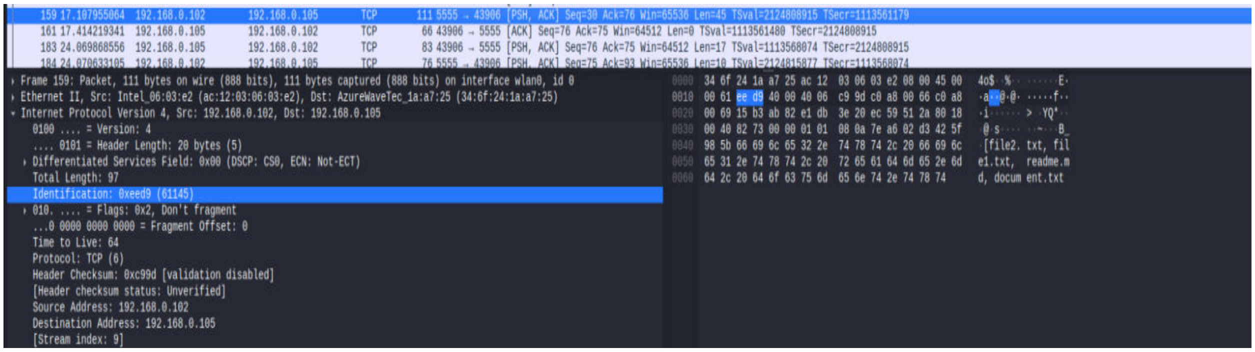




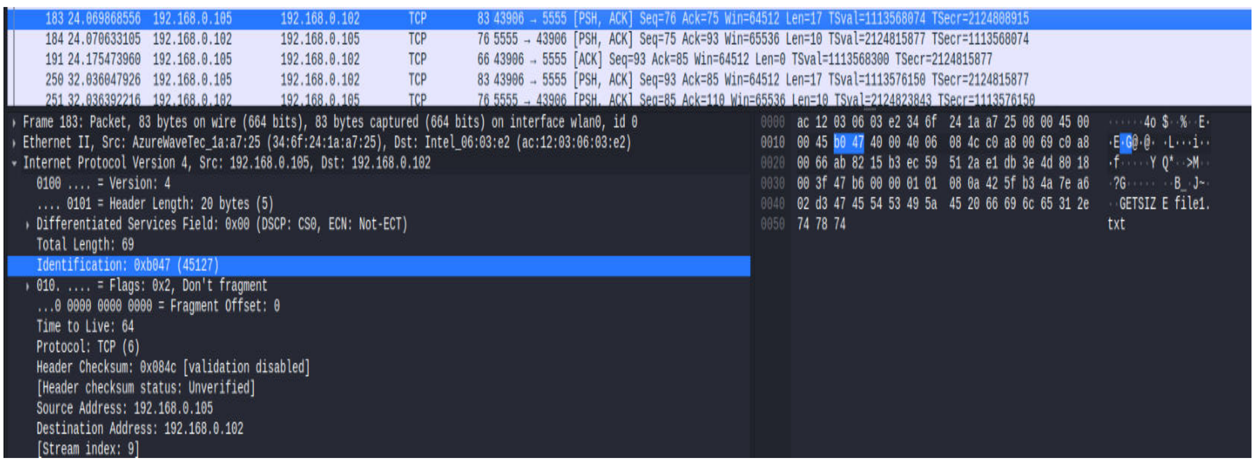
This capture shows that client (192.168.0.105) has requested a **LIST** command to know the list of directory from server(192.168.0.102).



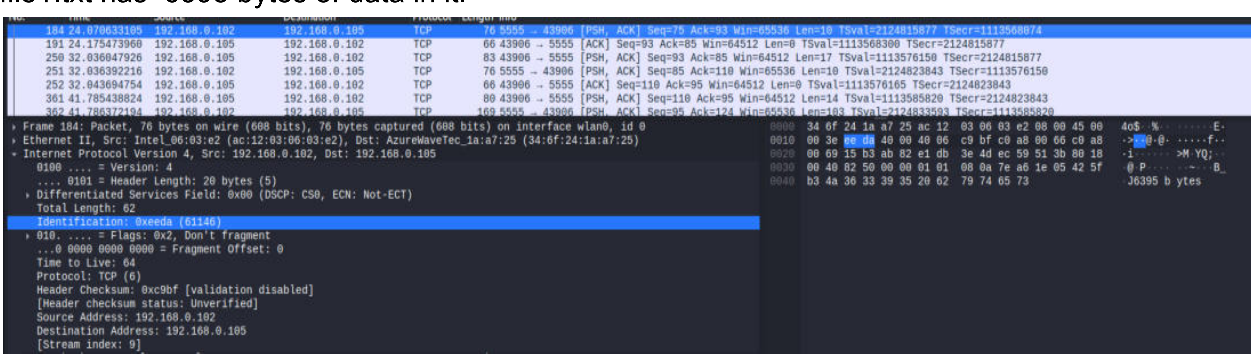
This capture shows that server(192.168.0.102) has given a response to client (192.168.0.105) for the LIST command as **RESOPNSE: file2.txt,file1.txt, readme.md**,**document.txt.**
This tells that the directory has these four files contained in its directory.



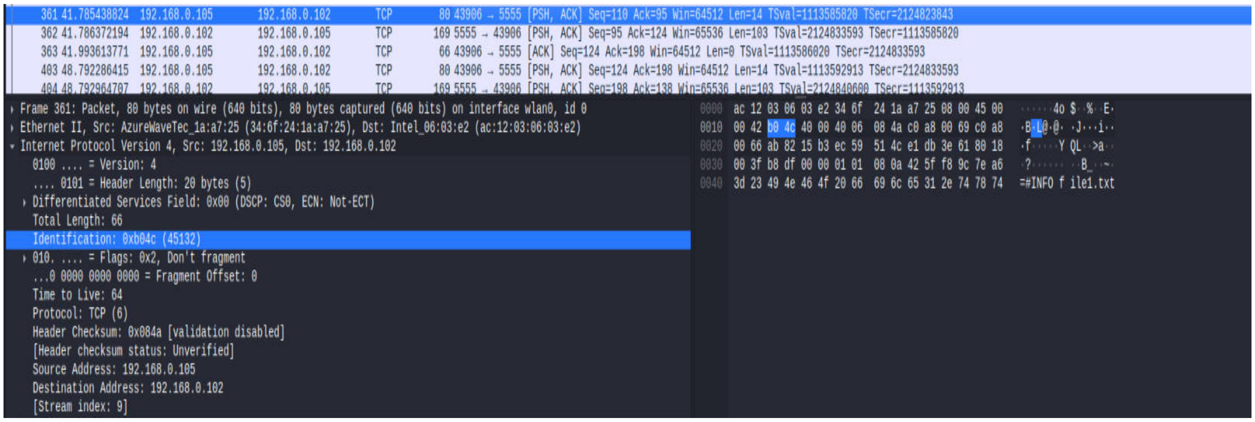
This capture shows that client (192.168.0.105) has requested GETSIZE file1.txt command to know the size of file1.txt from server(192.168.0.102) in the form of bytes.



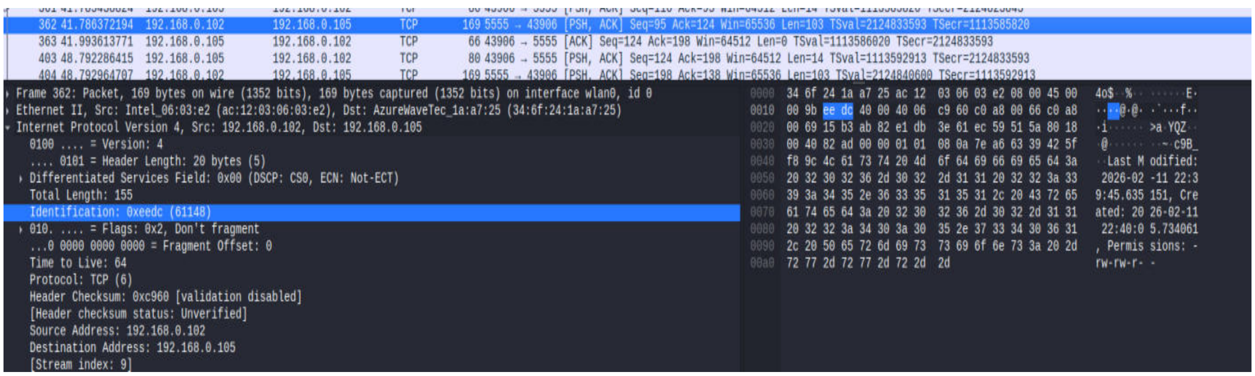
This capture shows that server(192.168.0.102) has given response to client (192.168.0.105) for the GETSIZE file1.txt command as **6395 bytes** which signify that complete file with name file1.txt has 6395 bytes of data in it.



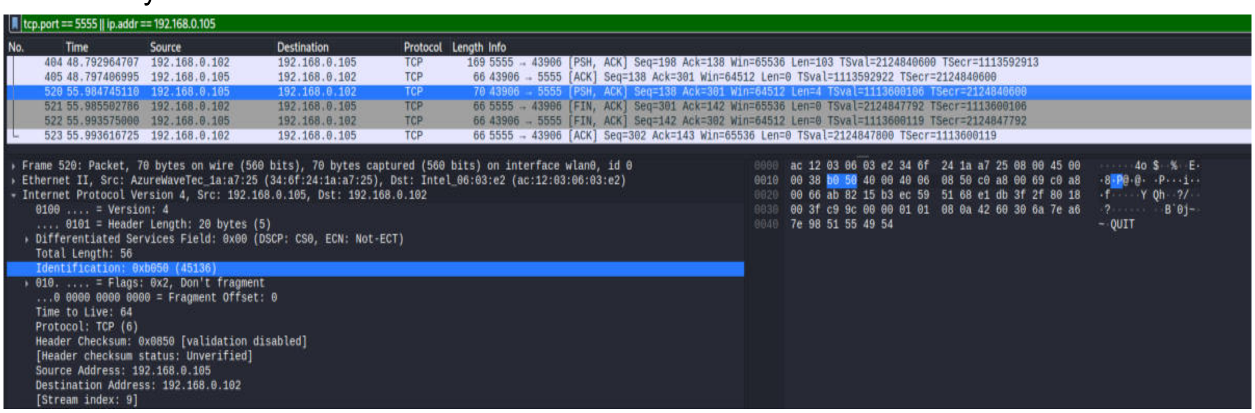
This capture shows that client (192.168.0.105) has requested INFO file1.txt command to know last modified timestamp as well as permissions of file1.txt from server(192.168.0.102)



This capture shows that server(192.168.0.102) has given response to client (192.168.0.105) for the INFO file1.txt command as shown in capture.



This capture of the terminal in Task 2 shows that client (192.168.0.105) has requested QUIT command to close the connection from server(192.168.0.102) after completion of the task successfully.



Task 3: File Download with Integrity

Summary and Design Overview

The goal of Task 3 is to ensure **Data Integrity** ,while Task 2 secured the session, Task 3 ensures that the files being downloaded are not tampered with or corrupted during transition.Here the "Chunked Transfer" mechanism where each 1024-byte block of data is verified using a Message Authentication Code (MAC) and tracked with a Sequence Number to prevent out-of-order delivery.

Implementation Approach

Based on the code provided, here is the technical breakdown:

File Chunking: The server reads the requested file in fixed blocks of **1024 bytes**. If the final block is smaller, it reads only the remaining bytes.

Sequence Numbering: A variable seq_no (starting at 0) is incremented for every chunk. This ensures that even if two chunks contain identical data, their MACs will be different, and the order is strictly enforced.

MAC Generation: For every chunk, the server calculates:

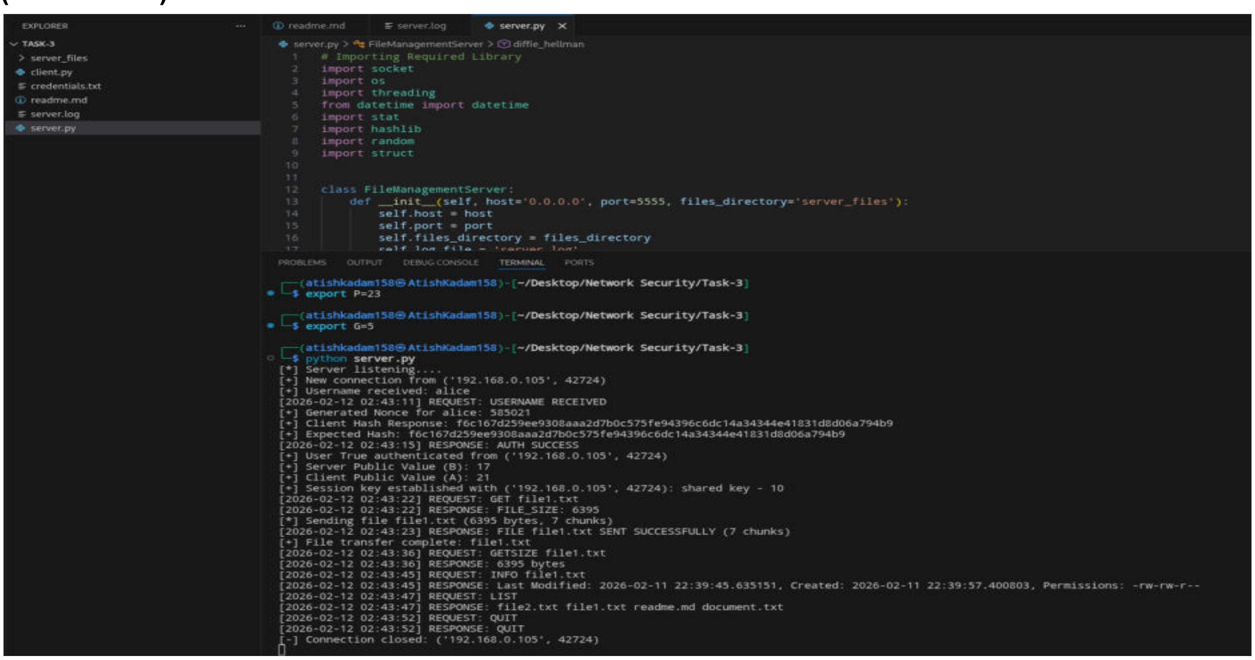
MAC = HASH(DATA || SEQ_NO || SESSION_KEY)

SERVER SIDE

This is the server.py file terminal capture

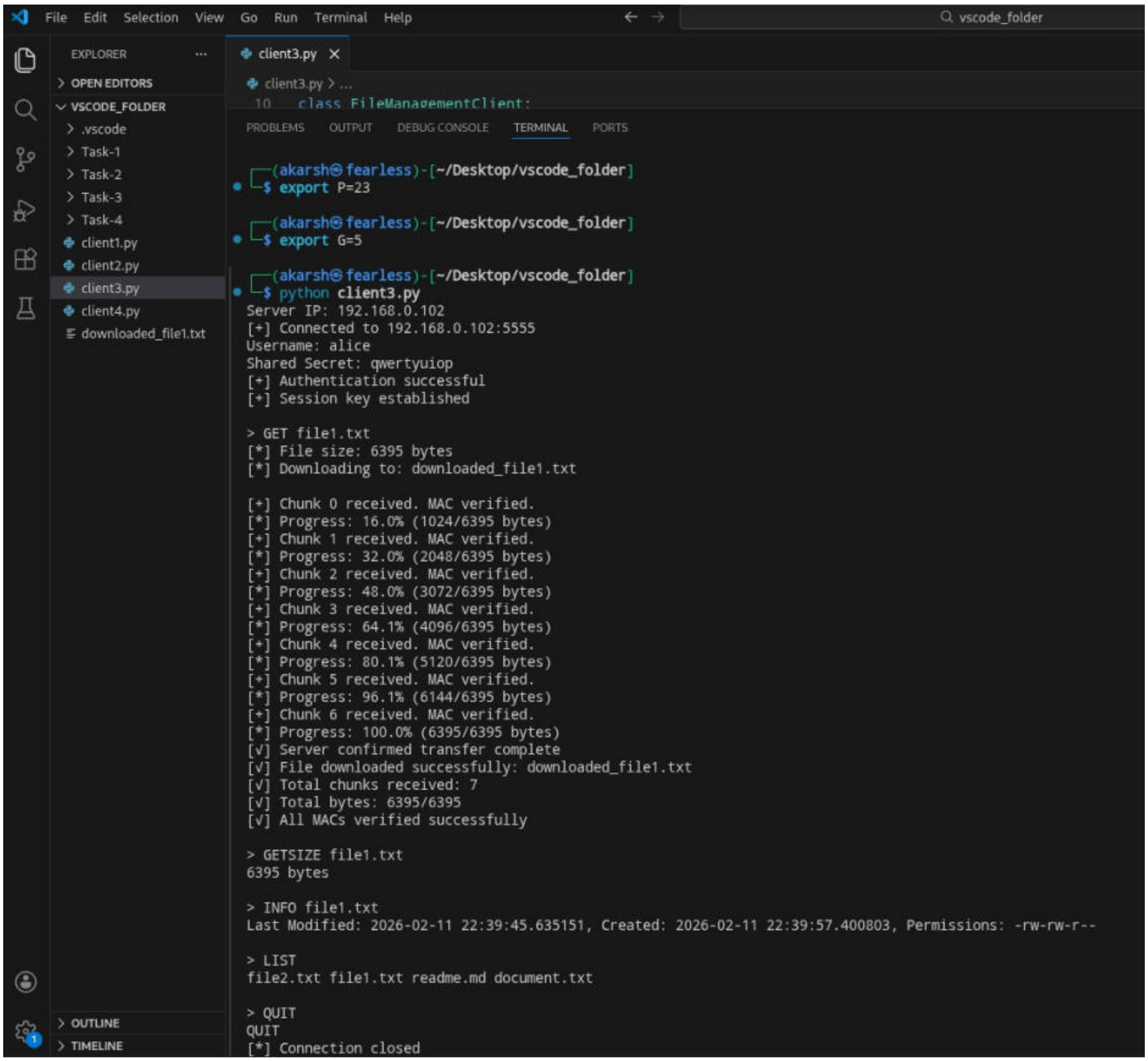
which shows how in TASK3 the file1.txt (total bytes of file1.txt is 6395) is transferred to client (192.168.0.102) in chunks of 1024 bytes

here in terminal capture of Server shows "**RESPONSE: FILE file1.txt SENT SUCCESSFULLY (7 CHUNKS)"** this shows the file is transferred to client.



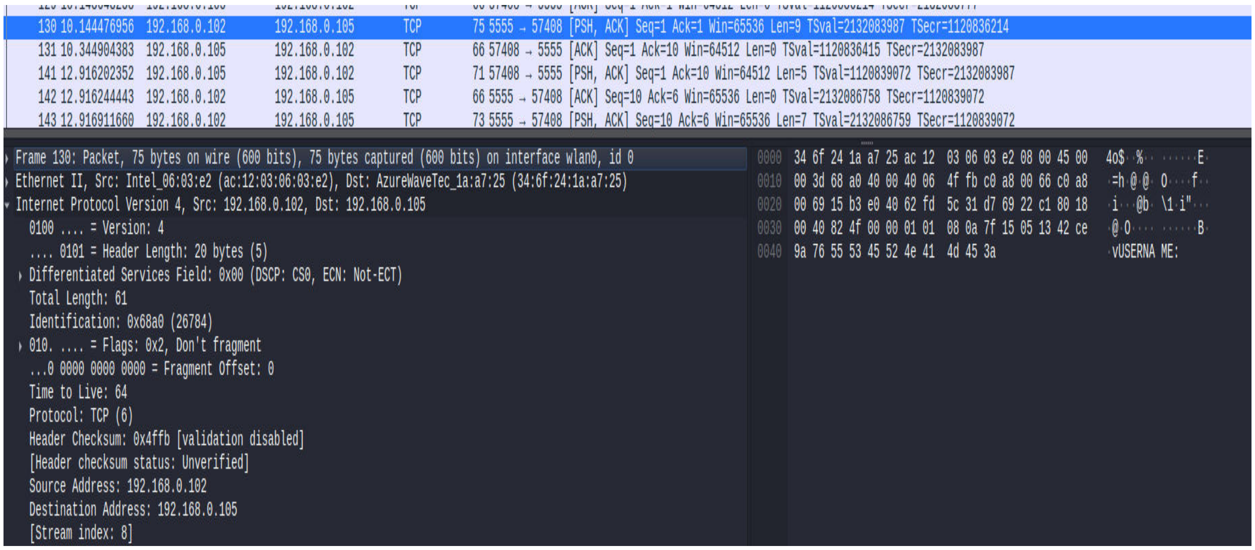
Client side

This is the client3.py file terminal capture
which shows how in TASK3 the file1.txt (total bytes of file1.txt is 6395) is received to client (192.168.0.102) in chunks of 1024 bytes
here in terminal capture of client shows
"**File downloaded successfully: downloaded_file1.txt**" this shows the file is transferred to client by server in the form of chunks 0 to chunk 6 along with MAC verified for each chunk.

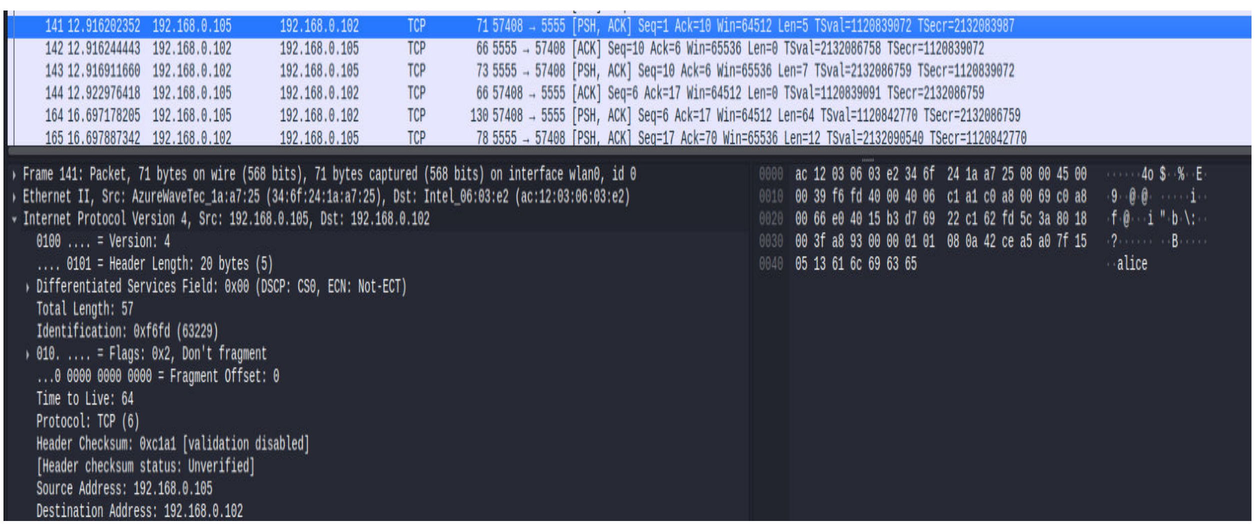


Now here is the wireshark capture of TASK 3

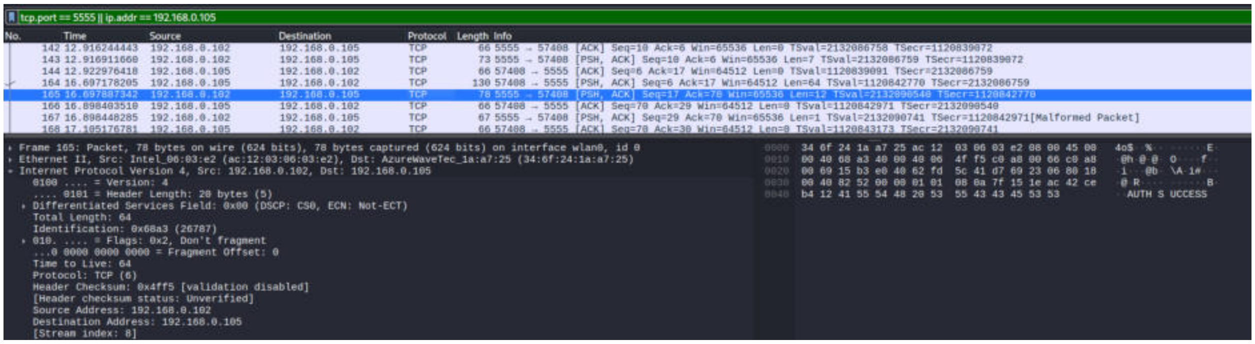
This is the wireshark capture of TASK 3 from server(192.168.0.102) to client (192.168.0.105) requesting for USERNAME from client after connected with user in order to check for valid authentication.



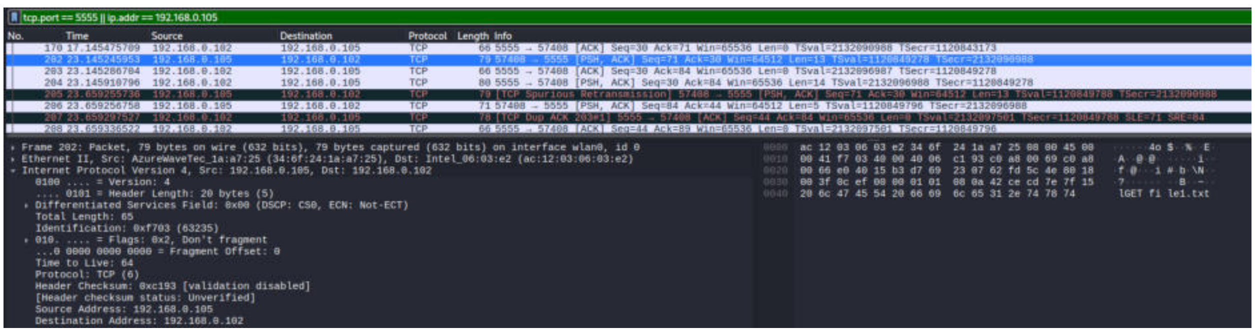
This is the wireshark capture of TASK 3 from client (192.168.0.105) to server(192.168.0.102) in which the user client is giving response of username as **"alice"** along with shared secret for authentication from server.



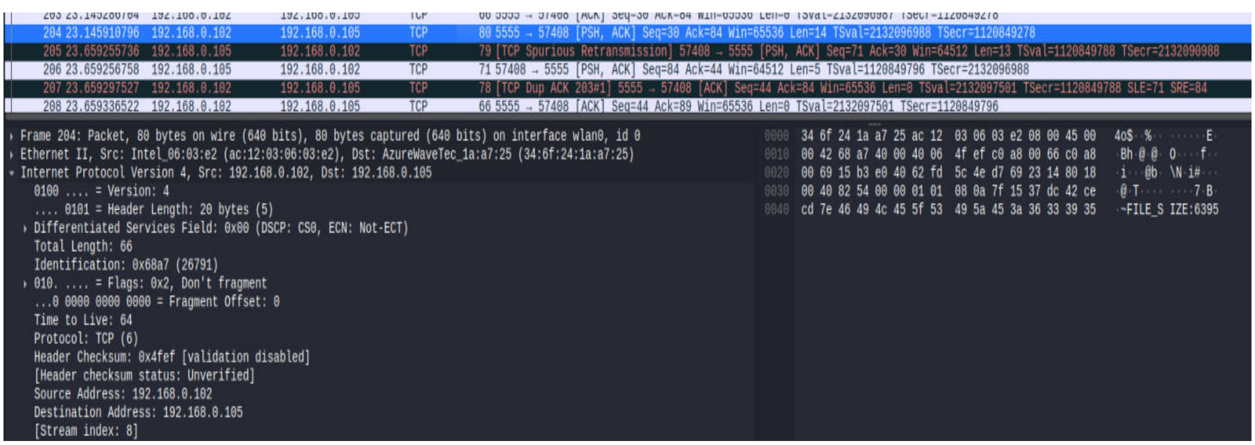
This is the wireshark capture of TASK 3 from server(192.168.0.102) to client (192.168.0.105) to tell that “AUTH SUCCESS” which means that the username credential is being authenticated by the server.



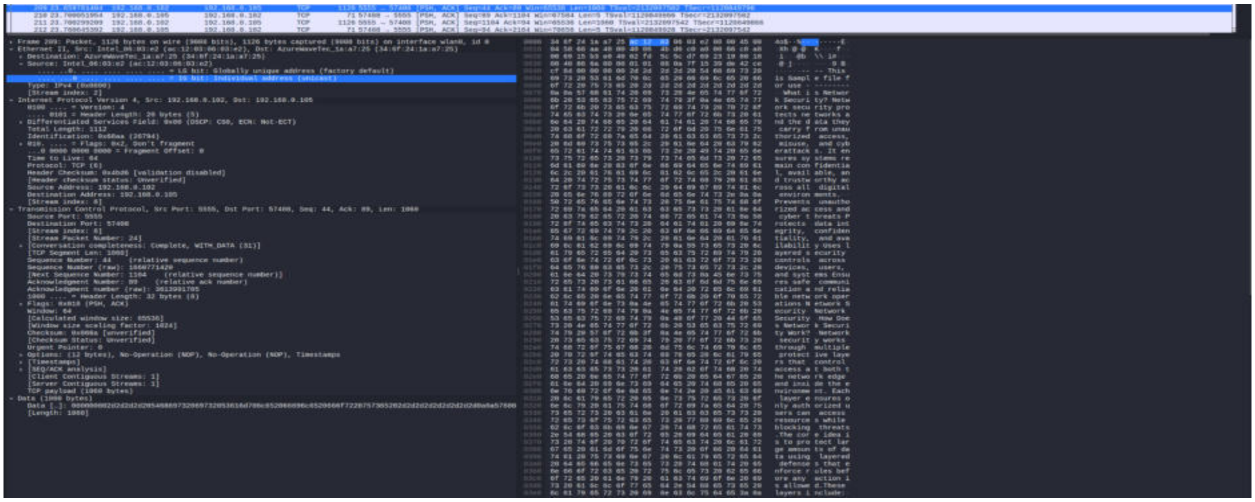
This is the wireshark capture of TASK 3 from client (192.168.0.105) to server(192.168.0.102) in which client requests to get the file1.txt from server .



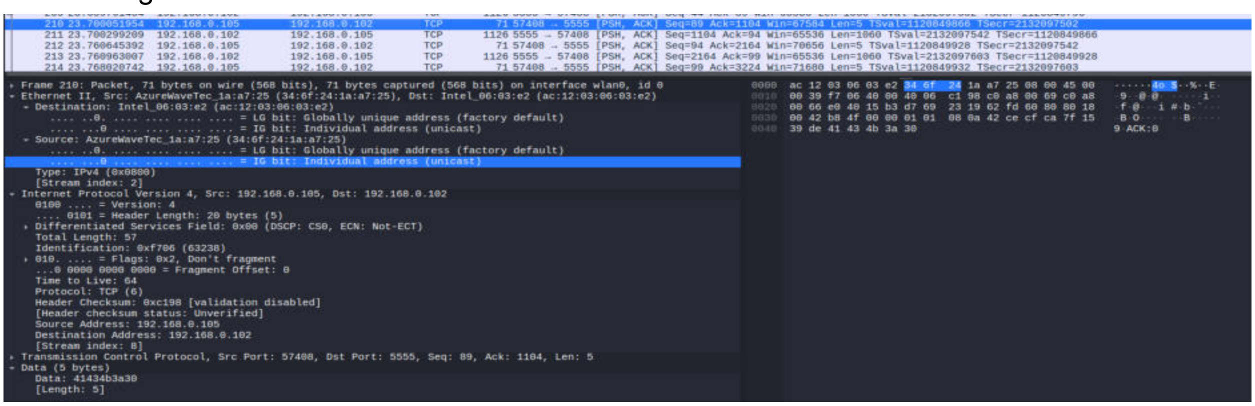
This is the wireshark capture of TASK 3 from server(192.168.0.102) to client (192.168.0.105) In which the server has sent the **FILESIZE of 6395** for the file1.txt to client ,later which the server will send the files in chunks to client .



Here the server has sent the file1.txt in chunks of 1024 bytes to the client .This wireshark capture shows about the chunk 0 of size 1024 bytes sent to the client.

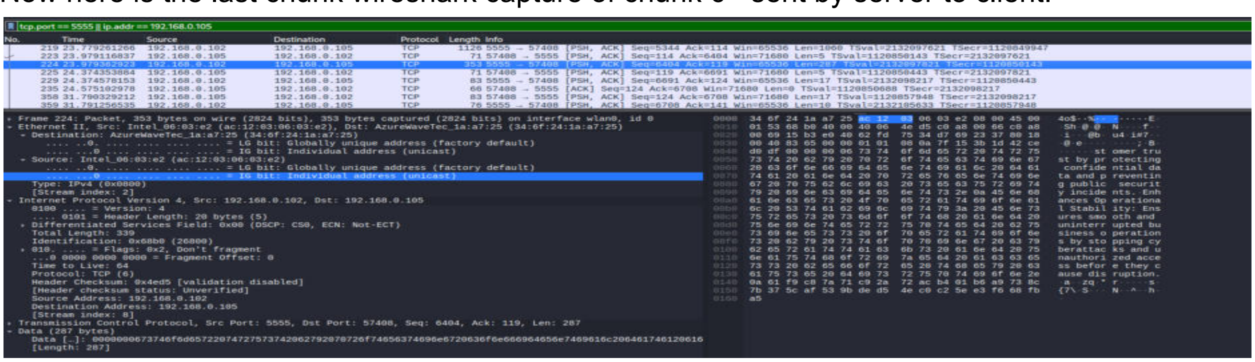


This capture of wireshark shows that chunk 0 has been received by client successfully and to acknowledge it the client has sent ACK 0 to the server .

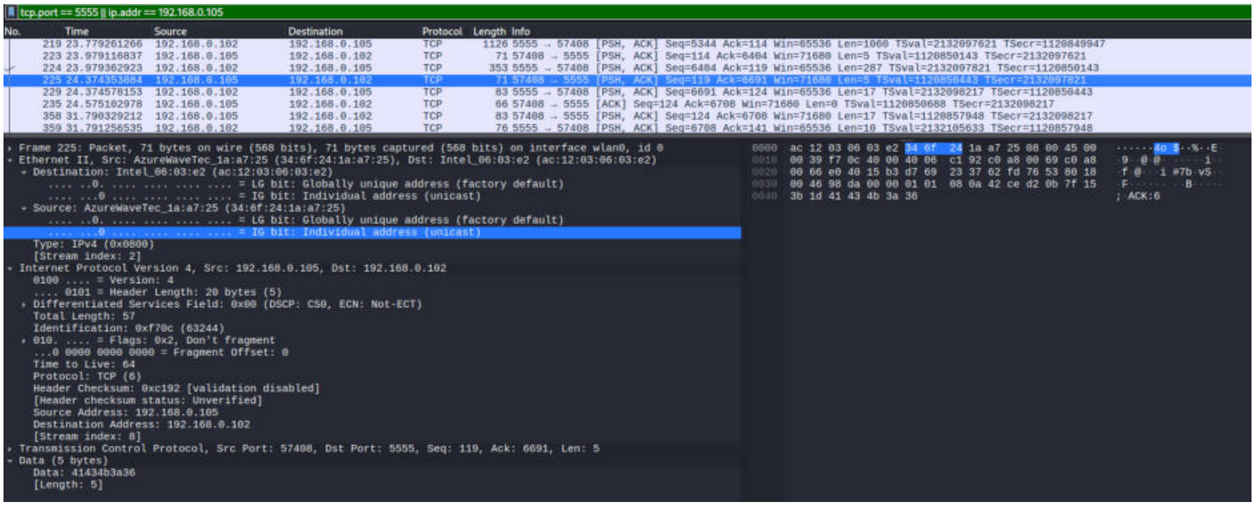


Similarly all chunks will show the similar type of capture in wireshark as per the content in the chunks.

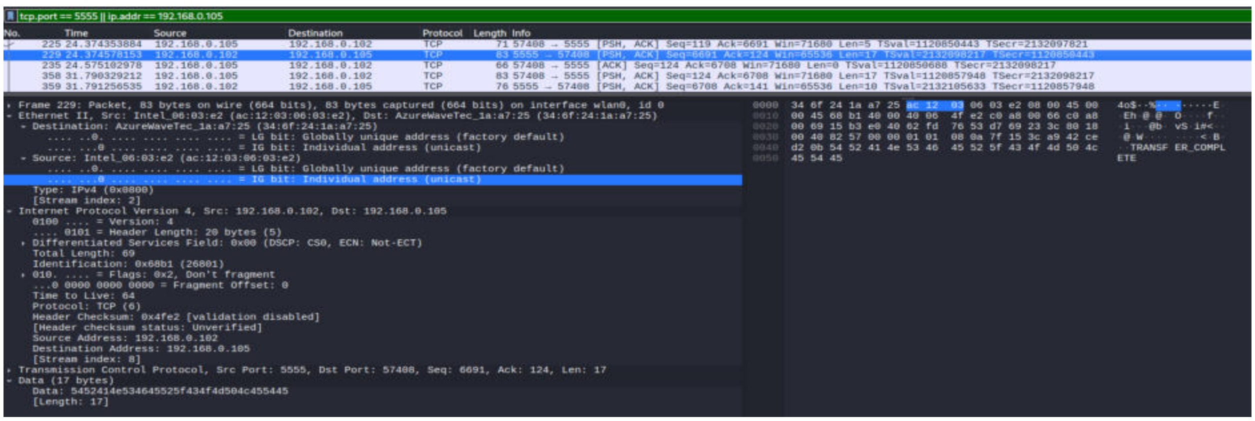
Now here is the last chunk wireshark capture of chunk 6   sent by server to client.



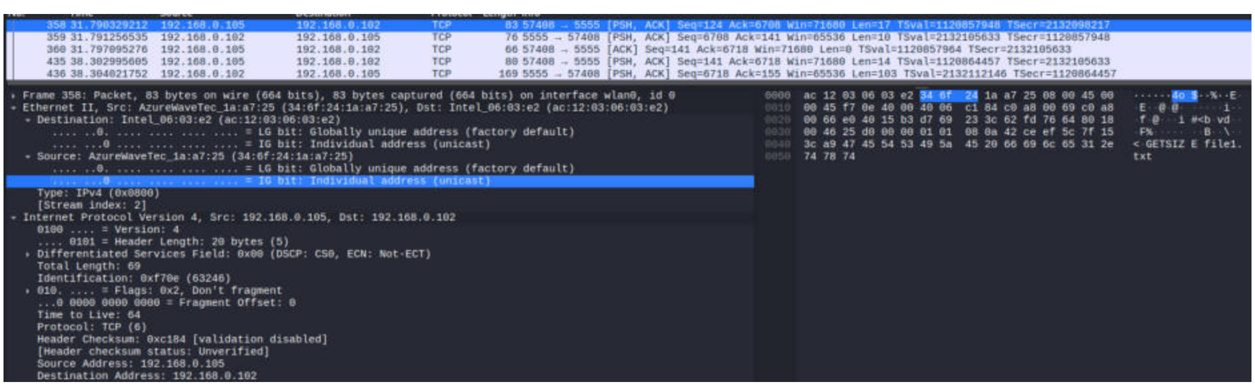
This is the wireshark capture of last chunk in which the client has acknowledged the last chunk 6 sent by the server as ACK 6.



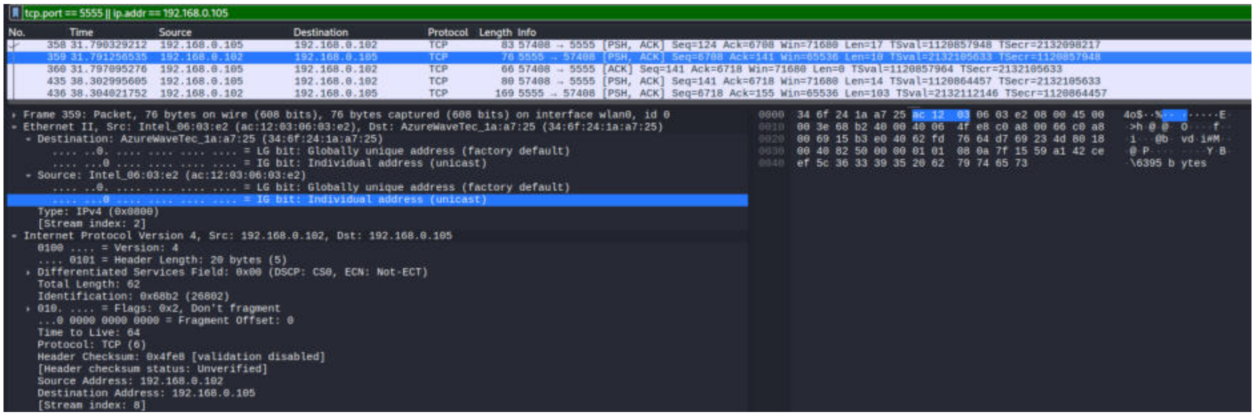
This confirmation from server to client after successful transfer of file in form of chunks as **FILE TRANSFER COMPLETE.**



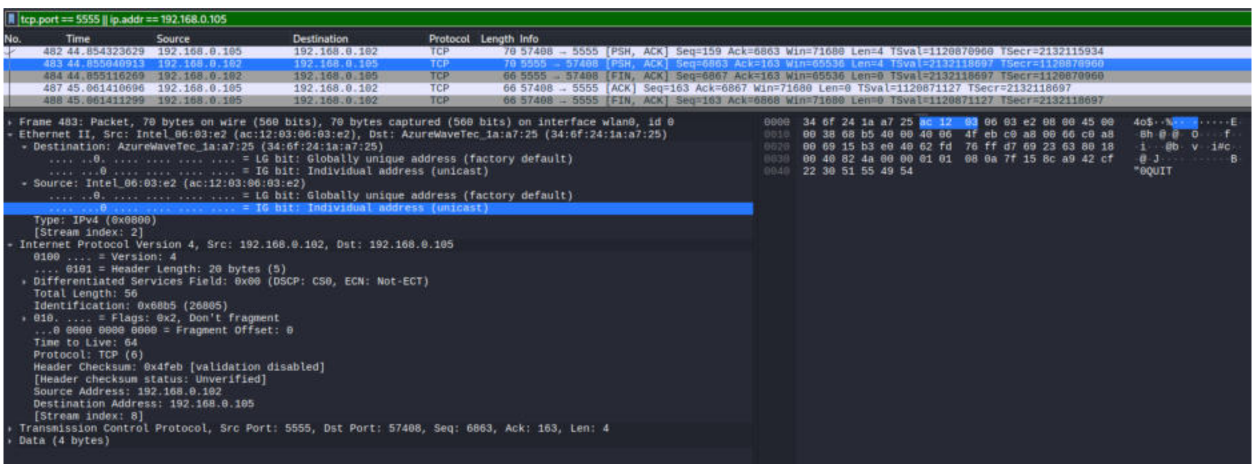
This is the capture of a request sent by client to server to know about the size of file1.txt using the command GETSIZE file1.txt.



This is the capture of response sent by server to client as **6395 bytes** ,which tells client about the size of file1.txt



This is the capture of wireshark which shows the response of the server as QUIT when the client has sent the request to close the connection with server.



Task 4: Encryption of Communication

Summary and Design Overview

For Task 4, the main goal was to move from just checking for tampering (integrity) to full-blown privacy (confidentiality). In the previous tasks, we made sure the session was safe, but here we're making sure that even if someone sniffs the network while `file1.txt` is being sent, they can't read a single word of it. I implemented **Authenticated Encryption**, which basically means every 1024-byte chunk is locked up tight using the AES key we generated during the handshake. This keeps the file private while also keeping the order-tracking from the previous tasks so the data doesn't get scrambled.

Implementation Approach

Looking at the terminal logs, you can see the process handled a 4988-byte file by breaking it into 5 chunks. Here's how I put it together:

- **Setting up the Keys:** First off, the client and server use that "qwertyuiop" shared secret to spin up a **Session Key and an AES Key**. Without this successful auth, the file transfer won't even start.
- **Chunk-by-Chunk Encryption:** I set the server to read the file in 1024-byte blocks. Before sending, each block is run through AES-256. In the screenshots, you can see the huge strings of hex code labeled **[Cipher]**—that's the raw encrypted data for things like Chunk 0 and Chunk 3.
- **On-the-fly Decryption:** Once the client gets a chunk, it uses the local AES key to decrypt it back into **[Plain]** text immediately. You can see the actual text about "Network Security" popping up in the logs as it decodes.
- **Double-Checking Metadata:** Before the download even begins, the client runs **GETSIZE** to confirm the file is exactly 4988 bytes. I also used an **INFO** command to pull the file permissions (`-rw-rw-r--`) and timestamps to make sure we're getting the right version of the file.
- **Final Verification:** The client tracks the count (Chunk 0 to 4) and only gives the **[v] Transfer Complete** signal once the total chunk count matches what was expected from the `GETSIZE` command.

Server Side

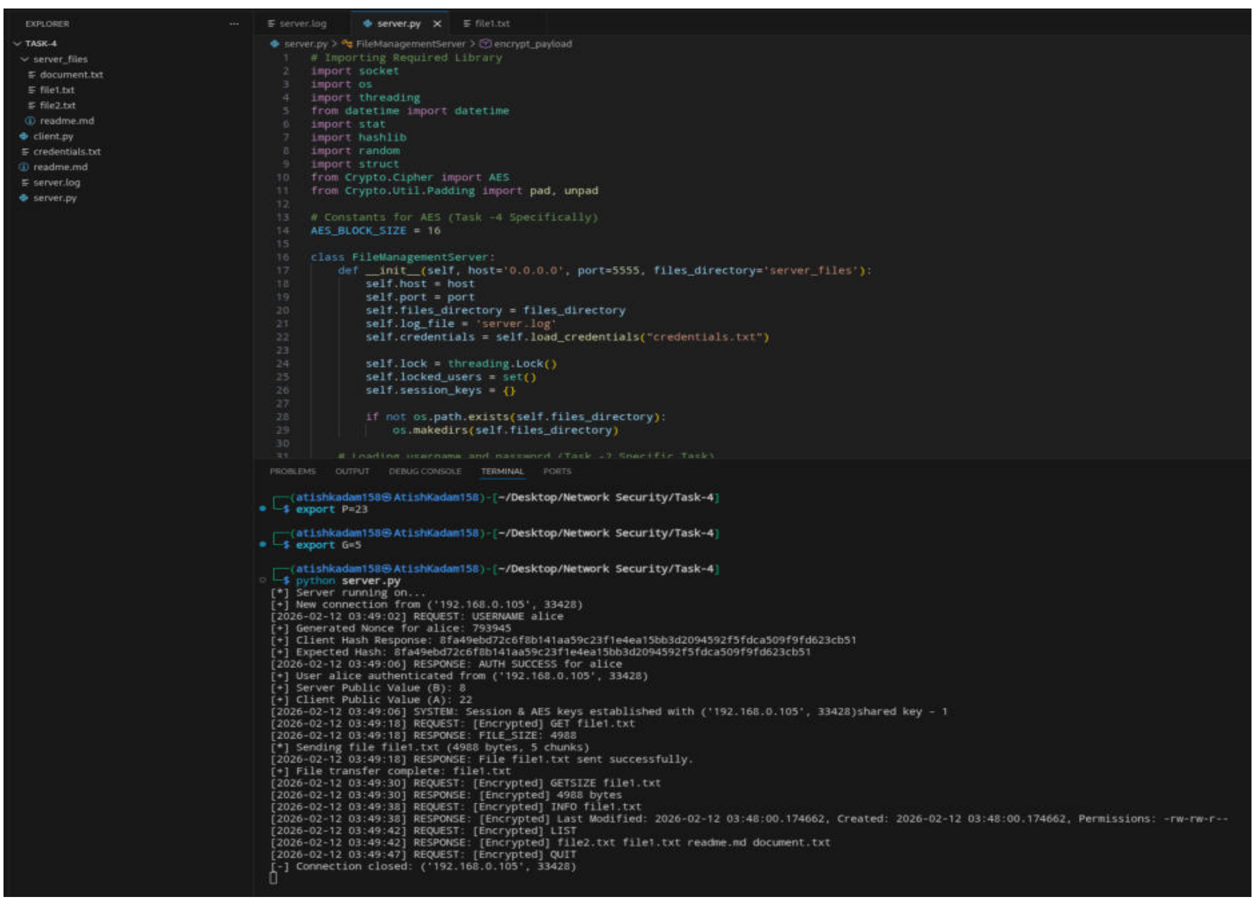
The server starts by exporting the necessary DH parameters (P=23,G=5) to set up the secure channel. Once the client connects, the server handles the handshake, verifying the username "alice" and the shared secret "qwertyuiop" to establish the AES session keys.

Command Processing: The images show the server responding to specific control commands like GET, GETSIZE, INFO, and LIST. For the GET command.

The Encryption: This is the main part of Task 4 is that the server reads file1.txt in chunks, and for each one, it generates that long string of "Cipher" text. It's essentially taking the plain text and encrypting it into a format that's unreadable without the session key before sending it.

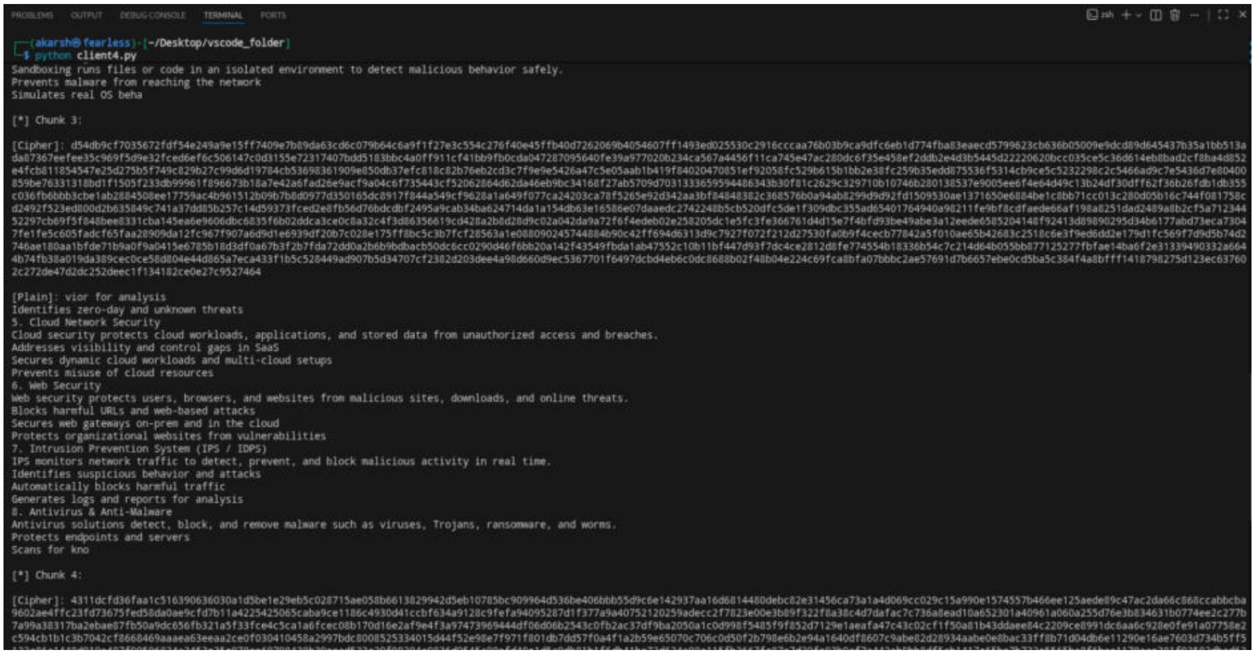
Metadata Handling: When the client asks for INFO or LIST, the server looks up the file system details (like the -rw-rw-r-- permissions and timestamps) and sends those back so the client can verify the file's state on the server side.

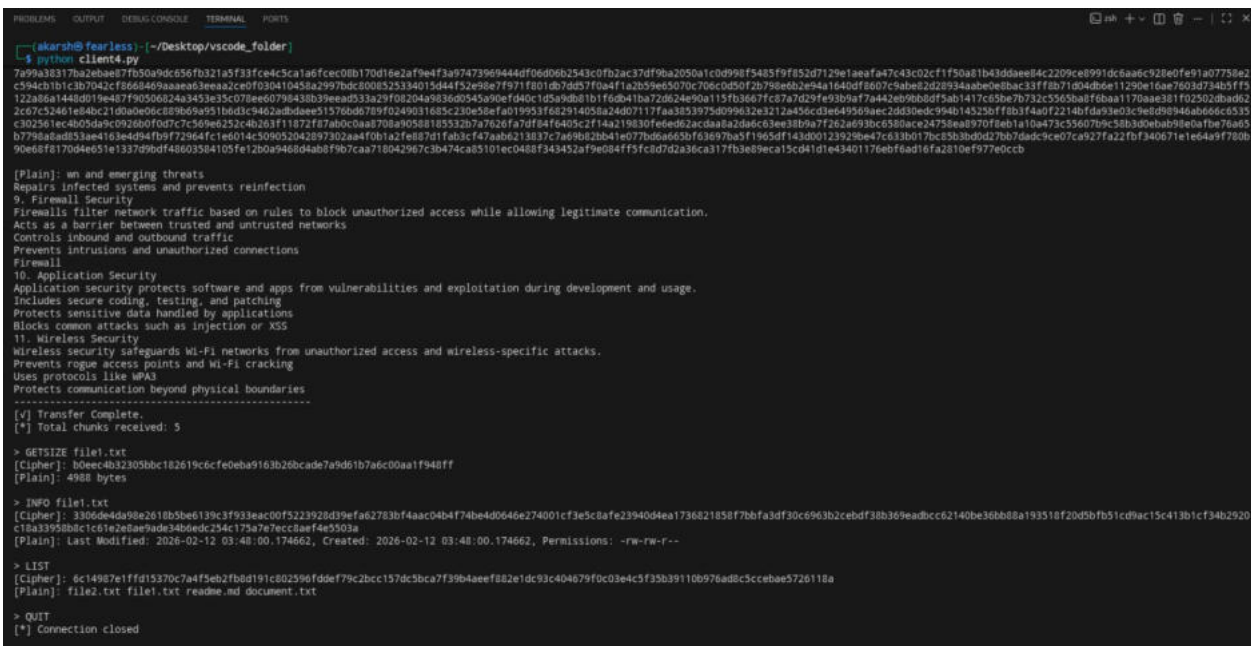
Graceful Exit: Finally, when the server receives the QUIT command, it closes the individual session and logs the "Connection closed" message, showing a clean end to the secure transaction.



Client side

Handshake & Setup: The client sets the DH parameters (P=23, G=5) and logs in as "alice" using the shared secret "qwertyuiop" to establish the AES session keys.
The Secure Download: After requesting file1.txt, the client confirms the 4988-byte size and begins pulling the data in 5 distinct chunks.
Real-time Decryption: The terminal displays the **[Cipher]** (encrypted hex) followed immediately by the **[Plain]** text. This proves the client is successfully using the AES key to decrypt the "Network Security" content as it arrives.
Metadata Verification: Post-download, the client uses GETSIZE, INFO, and LIST to verify the file's size, permissions (-rw-rw-r--), and directory status on the server.
Session Close: The client issues a QUIT command, resulting in a "Connection closed" message to end the secure session properly.

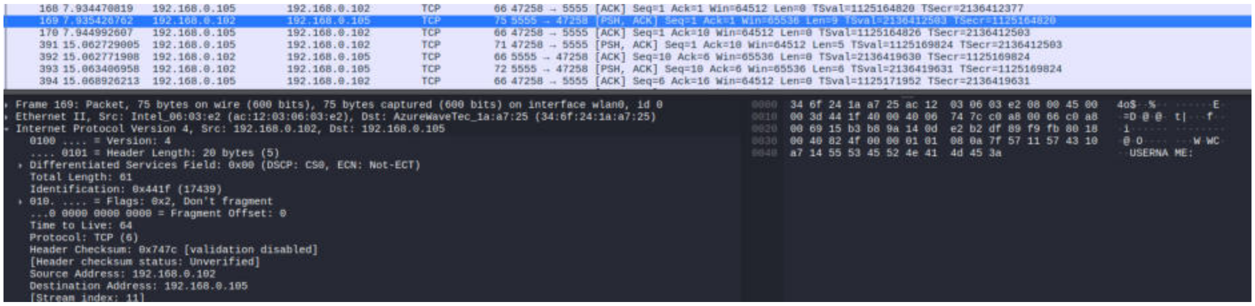

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(akarsh@fearless)-[~/Desktop/vscode_folder]
$ python client4.py
Sandboxing runs files or code in an isolated environment to detect malicious behavior safely.
Prevents malware from reaching the network
Simulates real OS beha
[*] Chunk 3:
[Plain]: vior for analysis
Identifies zero-day and unknown threats
5. Cloud Network Security
Cloud security protects cloud workloads, applications, and stored data from unauthorized access and breaches.
Addresses visibility and control gaps in SaaS
Secures dynamic cloud workloads and multi-cloud setups
Prevents misuse of cloud resources
6. Web Security
Web security protects users, browsers, and websites from malicious sites, downloads, and online threats.
Blocks harmful URLs and web-based attacks
Secures web gateways on-prem and in the cloud
Protects organizational websites from vulnerabilities
7. Intrusion Prevention System (IPS / IDPS)
IPS monitors network traffic to detect, prevent, and block malicious activity in real time.
Identifies suspicious behavior and attacks
Automatically blocks harmful traffic
Generates logs and reports for analysis
8. Antivirus & Anti-Malware
Antivirus solutions detect, block, and remove malware such as viruses, Trojans, ransomware, and worms.
Protects endpoints and servers
Scans for kno
[*] Chunk 4:


PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(akarsh@fearless)-[~/Desktop/vscode_folder]
$ python client4.py
[Plain]: wn and emerging threats
Repairs infected systems and prevents reinfection
9. Firewall Security
Firewalls filter network traffic based on rules to block unauthorized access while allowing legitimate communication.
Acts as a barrier between trusted and untrusted networks
Controls inbound and outbound traffic
Prevents intrusions and unauthorized connections
Firewall
10. Application Security
Application security protects software and apps from vulnerabilities and exploitation during development and usage.
Includes secure coding, testing, and patching
Protects sensitive data handled by applications
Blocks common attacks such as injection or XSS
11. Wireless Security
Wireless security safeguards Wi-Fi networks from unauthorized access and wireless-specific attacks.
Prevents rogue access points and Wi-Fi cracking
Uses protocols like WPA3
Protects communication beyond physical boundaries
[√] Transfer Complete.
[*] Total chunks received: 5
> GETSIZE file1.txt
[Cipher]: b0eec4b32305bbc182619c6cfe0eba9163b26bcade7a9d61b7a6c00aa1f948ff
[Plain]: 4988 bytes
> INFO file1.txt
[Plain]: Last Modified: 2026-02-12 03:48:00.174662, Created: 2026-02-12 03:48:00.174662, Permissions: -rw-rw-r--
> LIST
[Cipher]: 6c14987e1ffd15370c7a4f5eb2fb8d191c802596fddef79c2bcc157dc5bca7f39b4aeef882e1dc93c404679f0c03e4c5f35b39110b976ad8c5ccebae5726118a
[Plain]: file2.txt file1.txt readme.md document.txt
> QUIT
[*] Connection closed

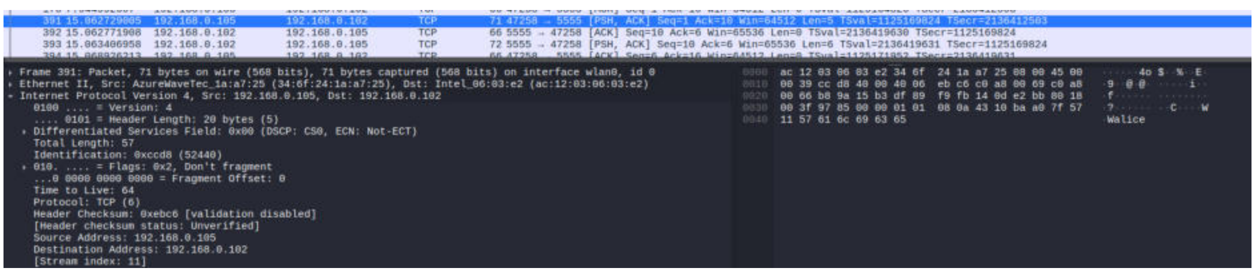
Now this is the Wireshark for TASK 4

This capture shows the initial TCP three-way handshake and the start of the application-level communication between the client (192.168.0.102) and the server (192.168.0.105).

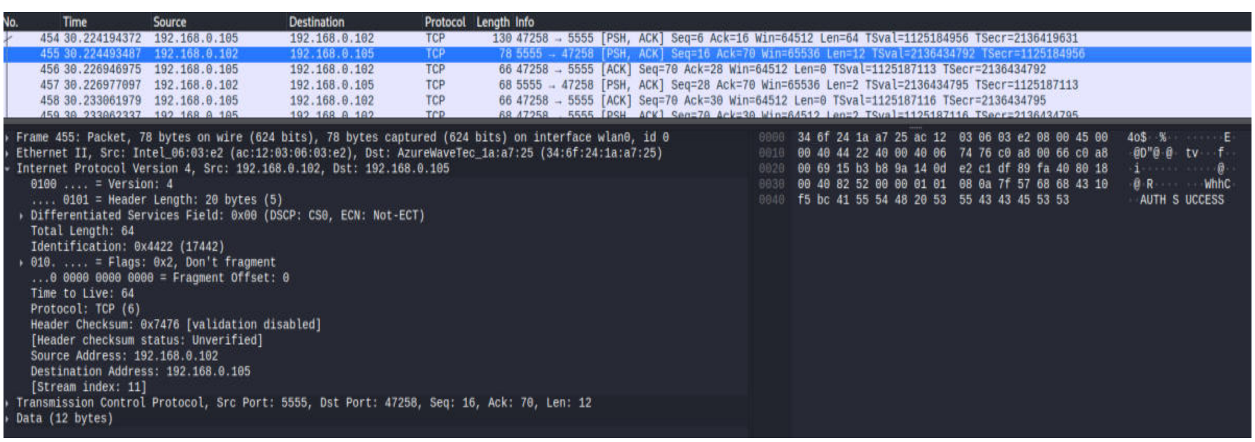
At this stage, the connection is established, and the client is identifying itself to the server.



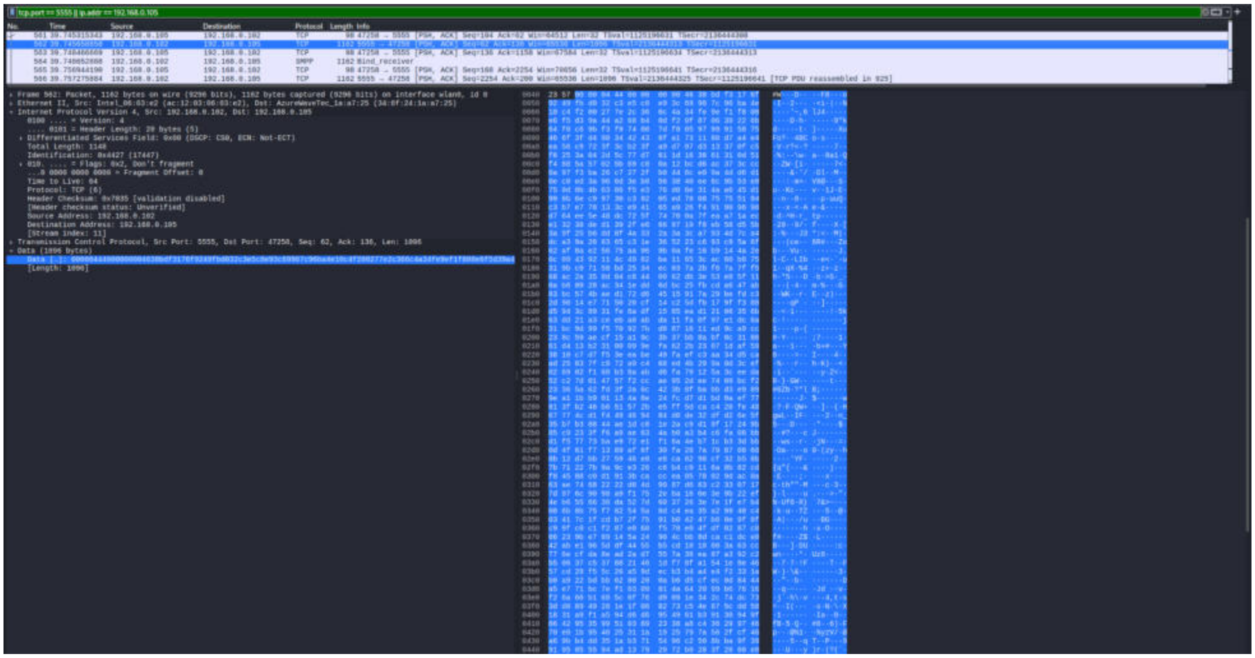
This capture indicates that the client is pushing the first set of data, which includes the **Username** as “**alice”** .   for authentication



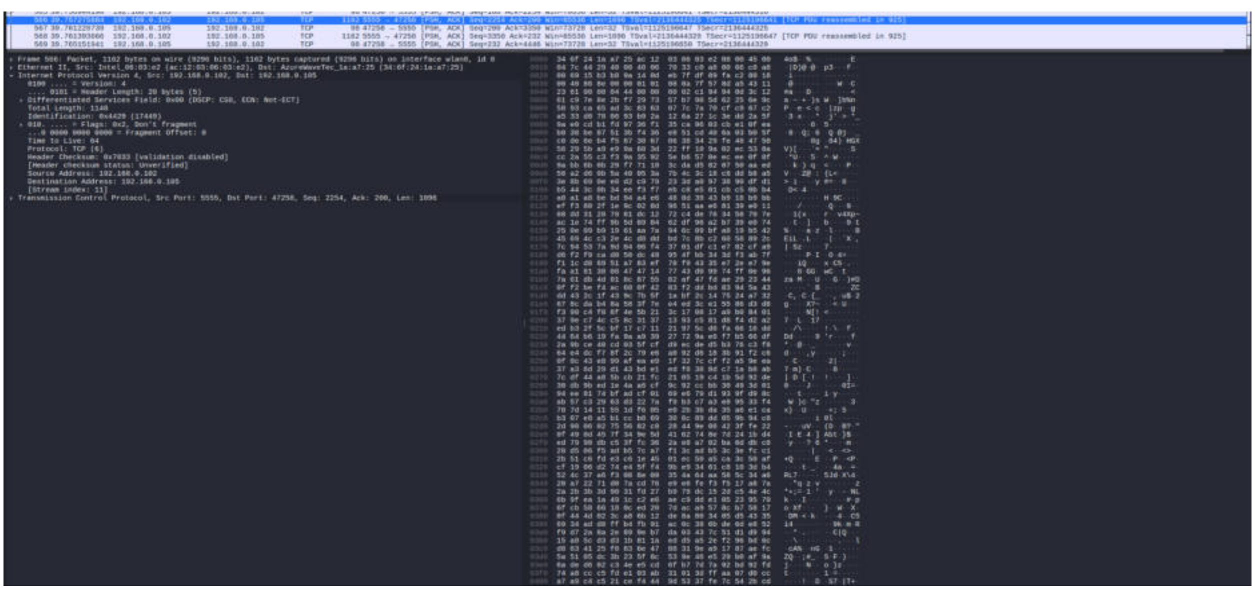
This snippet captures the critical moment of authentication and session key establishment
The hexadecimal data on the right decodes to **"AUTH SUCCESS"**. This confirms that the server has verified the shared secret and both parties have now derived the AES session keys needed for Task 4.



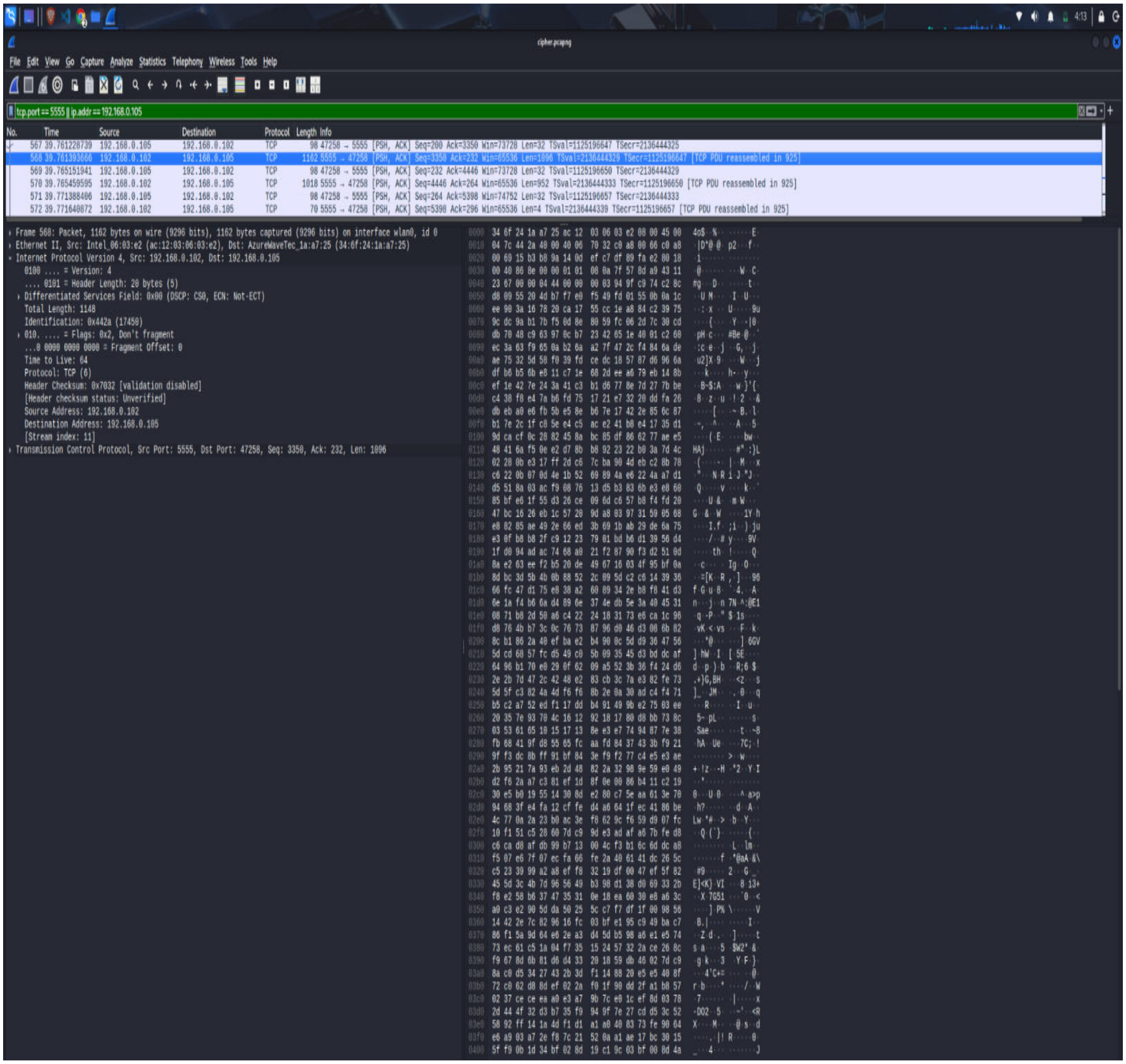
In this wireshark , this is a clear view of an active encrypted data transfer for **Chunk 0**
Unlike the previous authentication messages, the "Data" field here is completely unreadable in the ASCII preview.
This is the **[Cipher]** text in action ,it proves that even though Wireshark can see the packet, it cannot reveal the contents of the file being downloaded.



Similarly this is the clear view of an active encrypted data transfer for **Chunk 1** of file1.txt.
The **[Cipher]** text in action proves that even though Wireshark can see the packet, it cannot reveal the contents of the file being downloaded for chunk 1.

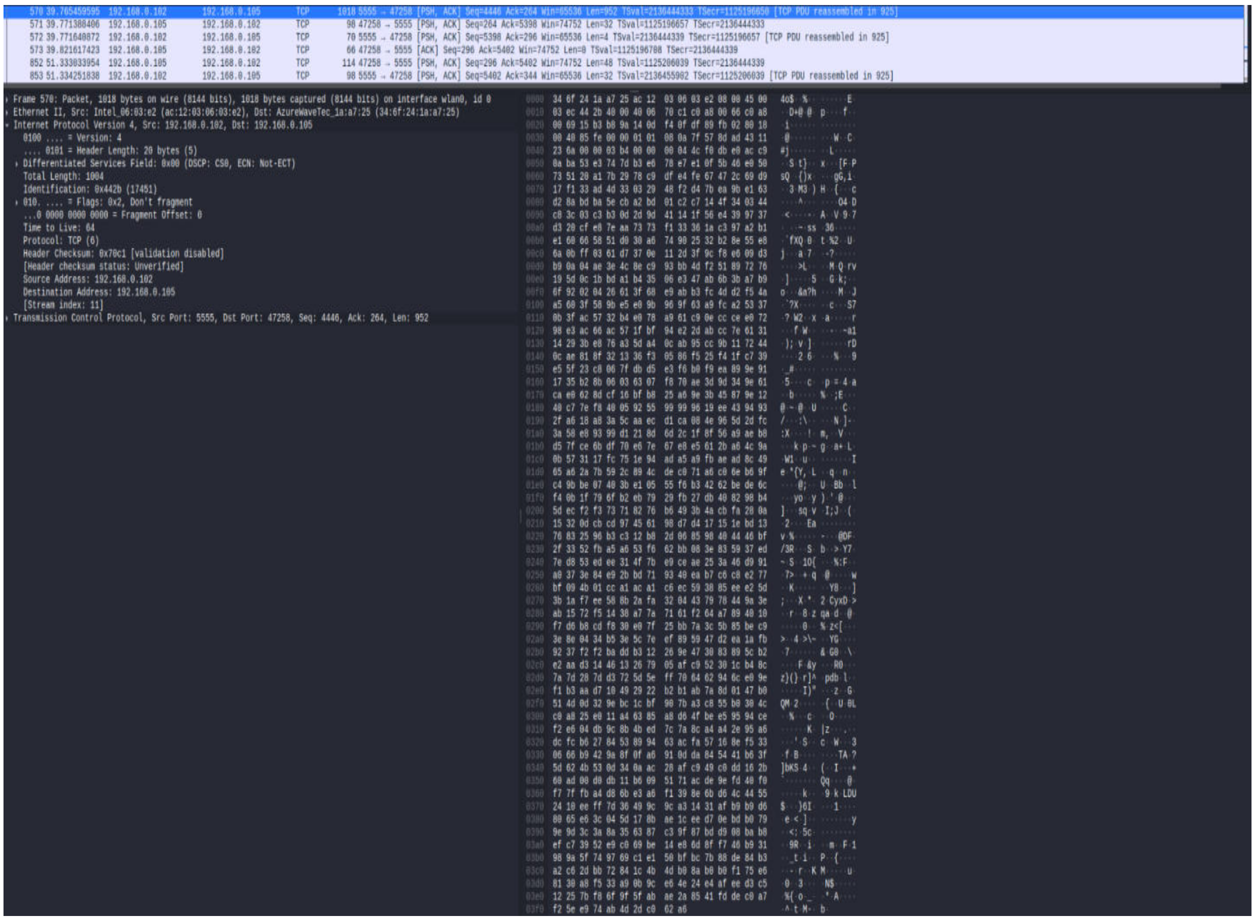


Similarly this is the clear view of an active encrypted data transfer for chunk 2 of file1.txt. The **[Cipher]** text in action proves that even though Wireshark can see the packet, it cannot reveal the contents of the file being downloaded for chunk 2



So in this way this last capture of a chunk of file1.txt shows the tail end of the file transfer eventual termination of the session.

The final data segments are sent before the client issues the QUIT command seen in the terminal images. The network traffic reflects the completion of the 4988-byte file transfer across multiple reassembled TCP segments.



ANTI-PLAGIARISM STATEMENT

We certify that this assignment/report is our own work, based on our personal study and/or research and that we have acknowledged all material and sources used in its preparation, whether books, articles,packages, datasets, reports, lecture notes, or any other document, electronic or personal communication.

We also certify that this assignment/report has not previously been submitted for assessment/project in any other course lab, except where specific permission has been granted from all course instructors involved, or at any other time in this course, and that we have not copied in part or whole or otherwise plagiarized the work of other students and/or persons.

We pledge to uphold the principles of honesty and responsibility at CSE@IITH. In addition, we understand my responsibility to report honor violations by other students if we become aware of it.

Names: **Akarsh Dubey**
Date: **14/02/2026**
Signature: Akarsh.

Names: **Atish Kadam**
Date: **14/02/2026**
Signature: Atish.